

DATA MINING & MACHINE LEARNING PROJECT

Credit Risk Assessment: Predicting Credit Defaulters

Prepared by:
Arman Zanganeh

An end-to-end supervised classification pipeline on 2,260,701 LendingClub loans — from raw data to a leakage-free

Dataset: LendingClub Accepted Loans (2007–2018Q4) | Task: Binary Classification (Default vs. Fully Paid) |
Champion Model: Optimized LightGBM

What This Presentation Covers

01

Motivation & Problem Definition

Why credit default prediction matters, and the challenges we expected going in

03

Data Preparation

Data cleaning & cleansing, Missingness policy, MICE imputation, encoding

05

Modeling, Tuning & Evaluation

Six candidate models, hyperparameter search, Testing ROC-AUC Score, **Testing Average Precision (AP)**

07

Final Discussion

What we learned, limitations, and what we'd do next

02

Data Understanding & Visualization

1.34M loans, class imbalance, correlation structure, raw distributions

04

The Leakage Investigation

Two rounds of data leakage found and fixed

06

Interpretability & Calibration

SHAP explanations and probability calibration

SECTION ONE

Motivation & Problem Definition

Why automated credit scoring matters, how we framed the target, and what we expected to be hard about this dataset before writing a single line of modeling code.

Why Credit Default Prediction Matters

- In peer-to-peer lending platforms like LendingClub, accurately separating creditworthy applicants from likely defaulters is the core risk-management function of the business.
Type I Error (False Positive): rejecting a good borrower — lost revenue and customer dissatisfaction.
Type II Error (False Negative): approving a borrower who later defaults — direct, severe capital loss.
- Goal: combine Gradient Boosting Trees with SHAP to build a scoring engine that is both accurate and auditable — able to explain why a specific credit decision was made.

Framing the Target Variable

- Task type: supervised binary classification. Learn $f(X) \rightarrow Y$ where $Y \in \{0, 1\}$.
- The raw loan_status field has 10 categories (Fully Paid, Current, Charged Off, Late, In Grace Period, Default, etc.).
- Ongoing loans (Current, Late, In Grace Period) are dropped — Current loans were excluded because they lack a definitive outcome, introducing ambiguity and noise into the binary classification model.
- Class 0 (Solvent): loan_status = Fully Paid.
- Class 1 (Defaulted): loan_status = Charged Off or Default.
- This filtering step alone removes 915,351 of the original 2,260,701 rows, leaving 1,345,350 conclusive, labeled observations to model.

Anticipated Data Challenges

Severe class imbalance — historically most borrowers repay in full, so defaults (the class we most need to catch) are a minority (~20%).

Data leakage — LendingClub records many fields that only exist after a loan is issued (late fees, recoveries, last payment amount, updated FICO scores). Left in the model, they leak the future into training.

High dimensionality & scale — 152 raw columns and 1.3M+ rows demand careful memory management, chunked processing, and disciplined feature selection rather than "throw everything at the model."

Missing data & noisy formats — many fields carry partial missingness with non-random patterns (MAR), and raw strings (e.g. "36 months to 36") need structured parsing before they are usable.

SECTION TWO

Data Understanding & Visualization

1.34 million labeled loans, 152 raw columns. Before touching the target we map the shape of the data: class balance, correlation structure, and how key financial variables split by outcome.

LendingClub Accepted Loans, 2007–2018



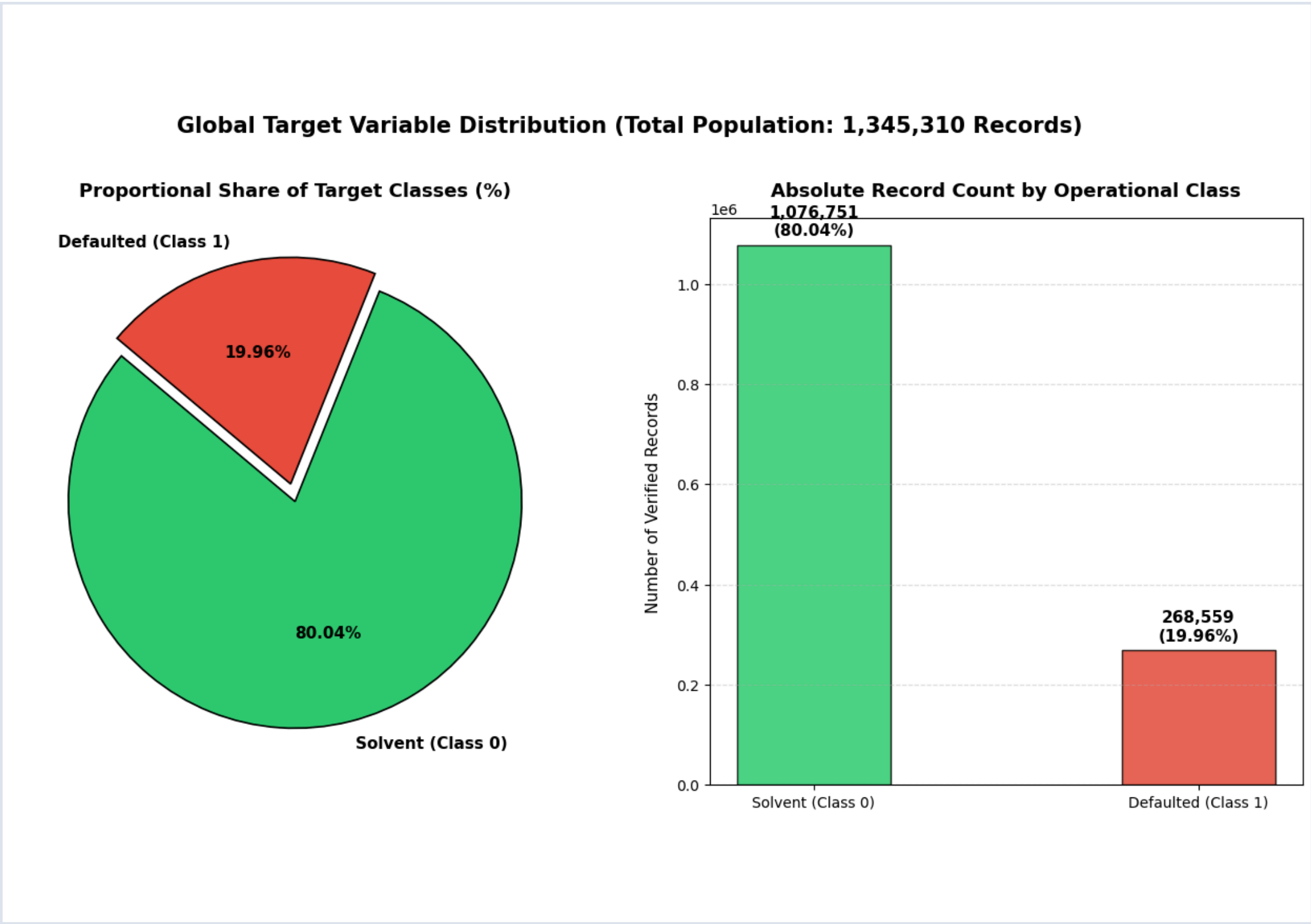
- Source: LendingClub's publicly released historical loan-level dataset (Kaggle mirror), covering originations from 2007 through Q4 2018.
Response variable: binary target derived from loan_status — 1,076,751 Fully Paid (Class 0) vs. 268,599 Charged Off / Default (Class 1).
Why this matters operationally: at roughly 1-in-5 base rate, a naive "always approve" policy would already be 80% "accurate" — which is exactly why accuracy alone is the wrong metric here, and why ranking-based metrics (ROC-AUC, Average Precision) drive this project.

Descriptive Statistics Audit (Raw, Uncensored)

Feature	Mean	Std Dev	Min	Median	Max (Raw)
loan_amnt (\$)	15,046.93	9,190.25	500	12,900	40,000
int_rate (%)	13.09	4.83	5.31	12.62	30.99
installment (\$)	445.81	267.17	4.93	377.99	1,719.83
annual_inc (\$)	77,992.43	112,696.20	0	65,000	110,000,000
dti (%)	18.82	14.18	-1.00	17.84	999.00
fico_range_low	698.59	33.01	610	690	845
revol_bal (\$)	16,658.46	22,948.31	0	11,324	2,904,836
revol_util (%)	50.34	24.71	0	50.30	892.30
total_acc	24.16	11.99	1	22	176
delinq_2yrs	0.31	0.87	0	0	58
mort_acc	1.56	1.90	0	1	94

113 numeric features were profiled at ingestion (2,260,668 raw rows, before target-based filtering). The raw max values here — annual_inc at \$110M, dti at 999%, revol_bal at \$2.9M — are the direct evidence motivating the winsorization step covered later in Data Preparation.

Class Distribution: A Structurally Imbalanced Problem

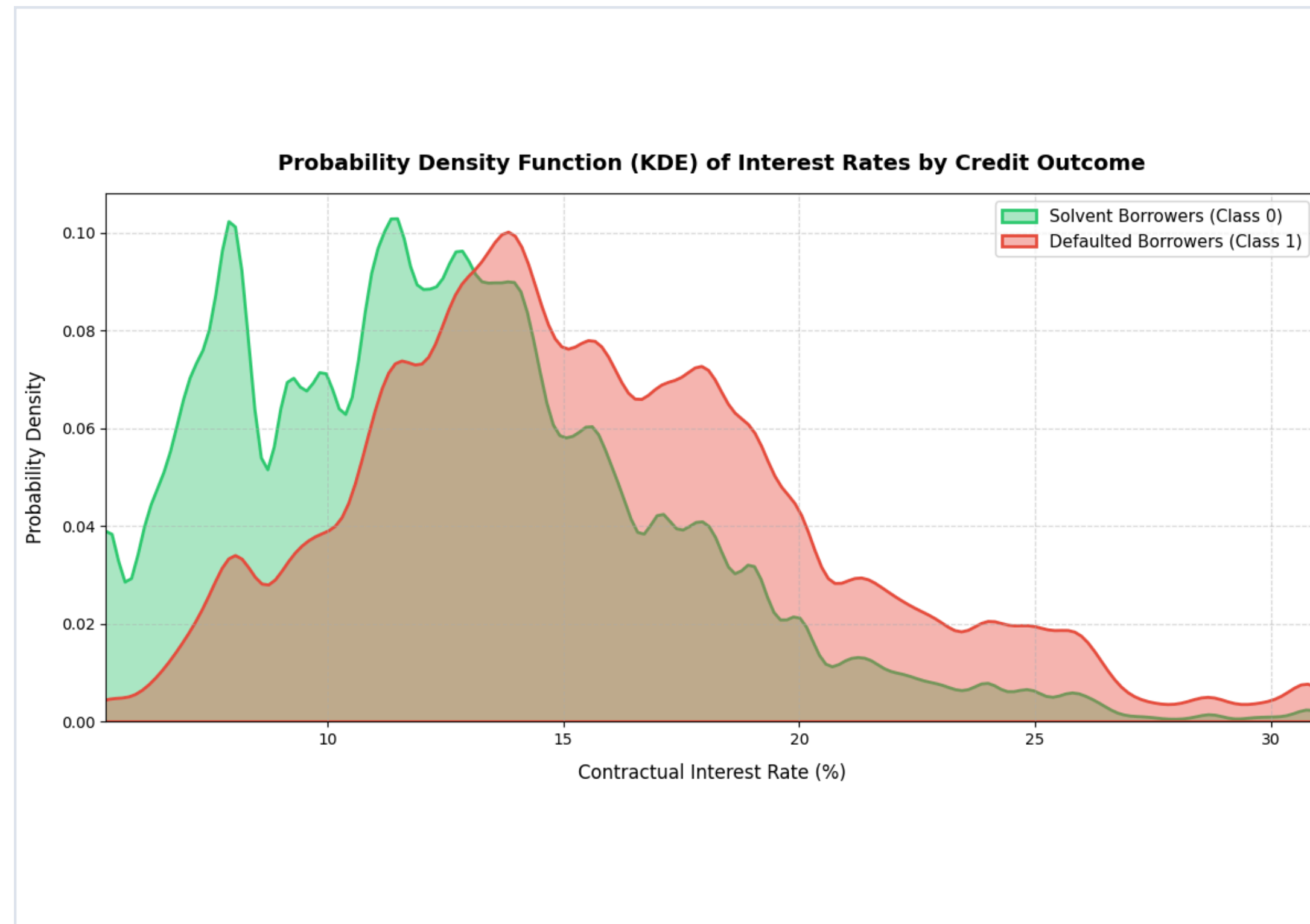


Anticipated Challenge #1: Class Imbalance

80.04% Solvent vs. 19.96% Defaulted across all 1,345,310 labeled records.

- This ~4:1 imbalance is a defining constraint for the whole project: it shapes the loss function (class weighting over synthetic resampling at this scale), the choice of evaluation metric (ROC-AUC and Average Precision instead of raw accuracy).
- **Why not SMOTE at this scale?** With 1.3M+ rows, synthetic oversampling is computationally expensive and offers little benefit over class-weighted boosting, which achieves the same rebalancing effect natively inside LightGBM/XGBoost.

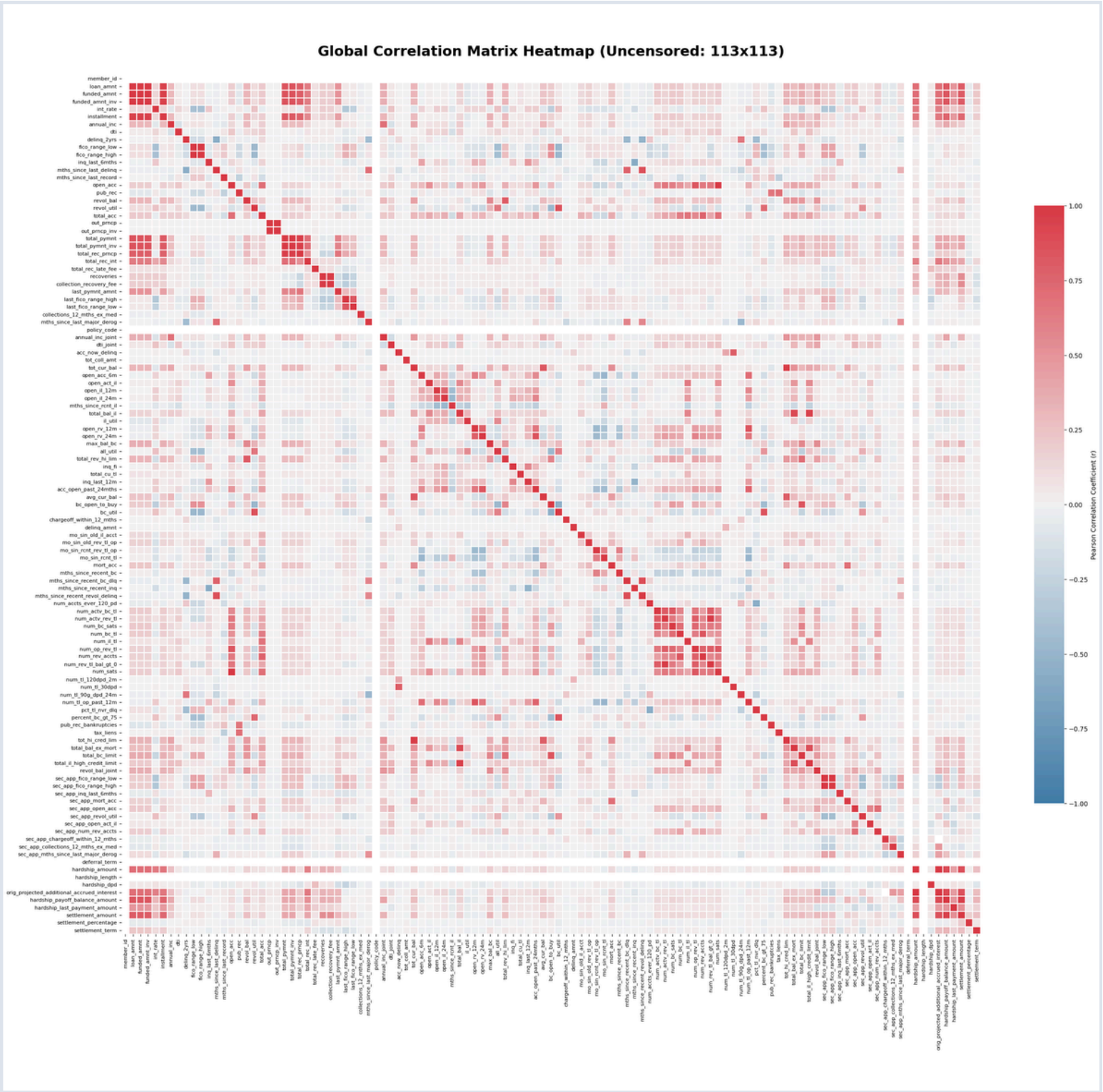
Interest Rate Distribution by Credit Outcome



- The KDE overlay shows solvent borrowers (green) concentrated at lower interest rates (~7–12%), while defaulted borrowers (red) skew toward higher rates (~14–20%) with a heavier right tail.
- This is expected and mechanically sound: LendingClub's own underwriting already prices perceived risk into the interest rate via sub-grade — so `int_rate` carries genuine, legitimate predictive signal rather than being a leakage artifact.
- The two distributions overlap substantially in the 10–15% band, confirming that interest rate alone is a weak-to-moderate discriminator — exactly what motivates a multivariate model rather than a single-feature cutoff rule.

Correlation Matrix Heatmap & Multicollinearity Diagnosis

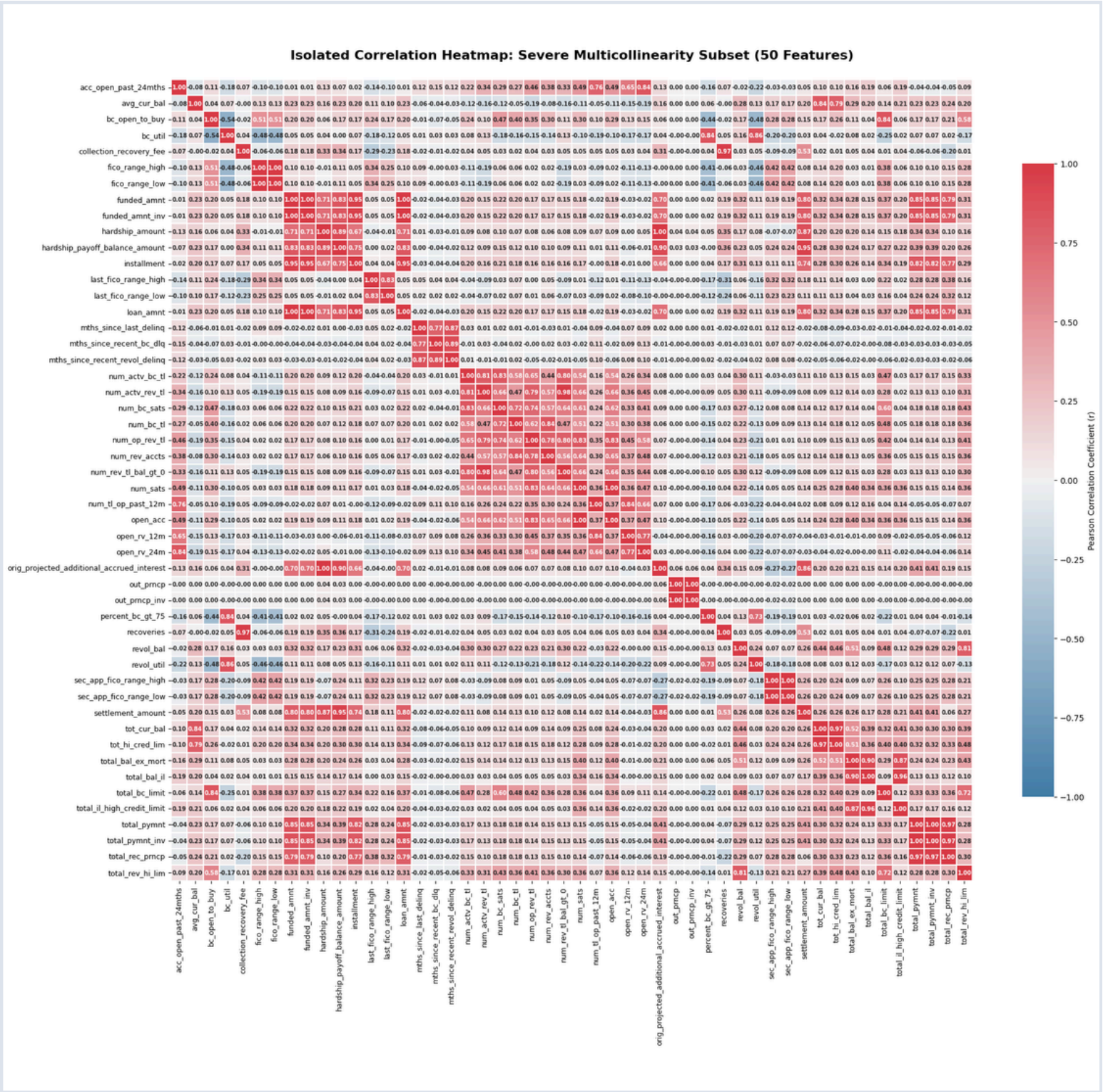
(113 Numeric Features, Raw)



Reading the matrix: dense red blocks mark tightly correlated feature families — loan amount variants (loan_amnt / funded_amnt / funded_amnt_inv), FICO range pairs, and revolving-credit utilization metrics.

Diagnostic finding: 55 feature pairs exceed $|r| > 0.80$, involving 50 distinct columns — evidence of substantial redundancy that must be pruned before modeling, both for interpretability and to stabilize tree-based feature importance.

Isolated View: High-Collinearity Feature Pairs ($|r| > 0.80$)



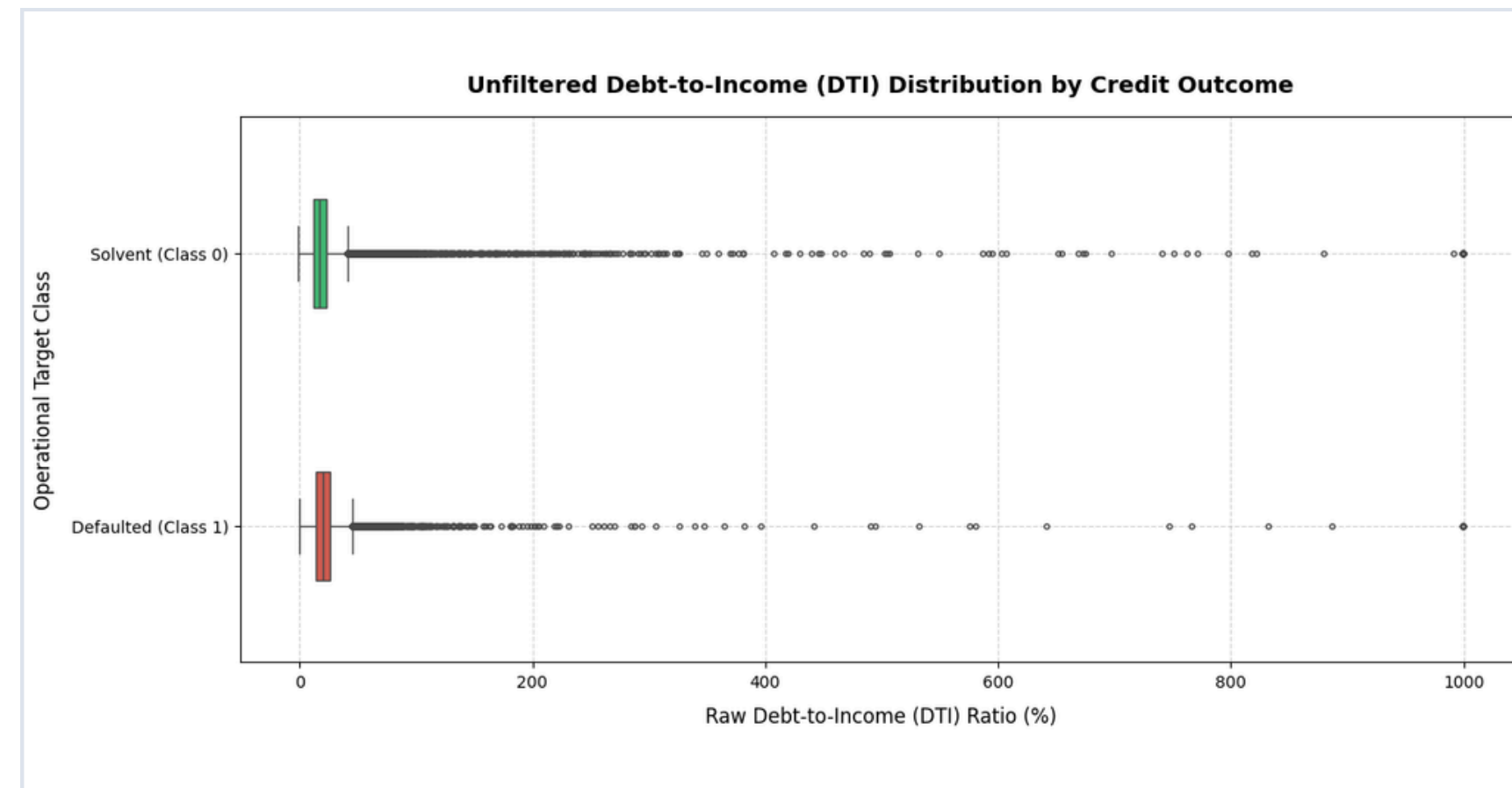
- Zooming into only the 50 columns implicated in high-collinearity pairs makes the redundant families explicit: installment/loan amount variants, FICO low/high range pairs, and revolving balance/limit metrics each form near-perfect correlation blocks.
- Action taken downstream:** rather than dropping features by hand, Phase 3.4 later runs an automated single-variable logistic AUC comparison on every pair above 0.95 correlation and keeps only the stronger predictor of each pair.

High-Correlation Pairs Registry (Representative Sample of 55)

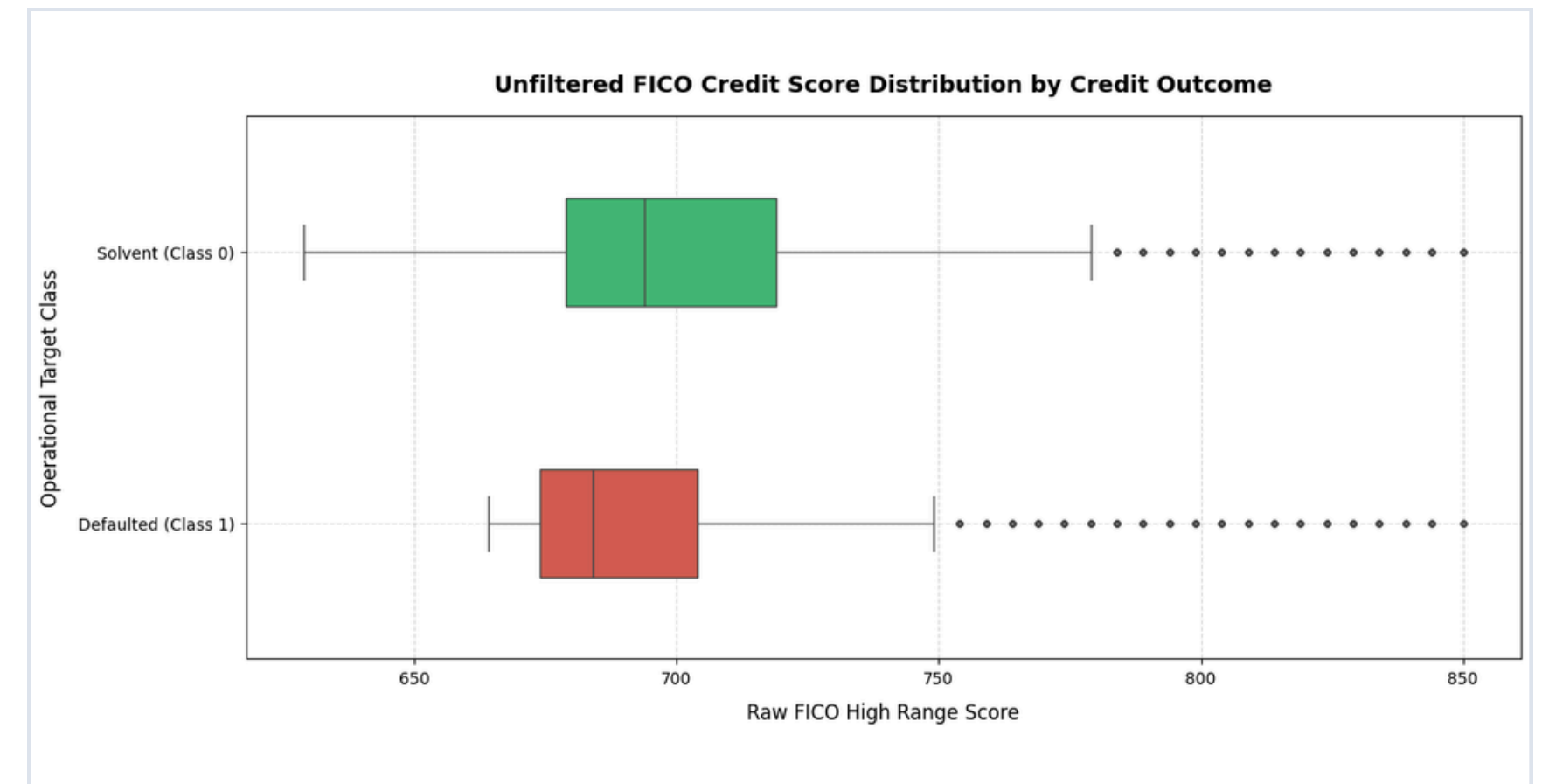
Full automated scan identified 55 pairs exceeding $|r| > 0.80$ across 50 distinct columns, spanning six feature families. This sample illustrates the breadth; every pair was resolved in Data Preparation via missingness pruning or AUC-based collinearity filtering.

Variable A	Variable B	$ r $	Feature Family
loan_amnt	funded_amnt	0.9996	Loan amount variants
loan_amnt	funded_amnt_inv	0.9986	Loan amount variants
funded_amnt	installment	0.9541	Loan amount vs. installment
fico_range_low	fico_range_high	1.0000	FICO bureau pair
out_prncp	out_prncp_inv	1.0000	Outstanding principal pair
total_pymnt	total_pymnt_inv	0.9993	Total payment pair
total_pymnt	total_rec_prncp	0.9672	Payment vs. principal recovered
recoveries	collection_recovery_fee	0.9705	Post-origination collections
last_fico_range_high	last_fico_range_low	0.8295	Retrospective FICO pair
revol_bal	total_rev_hi_lim	0.8102	Revolving credit exposure
tot_cur_bal	avg_cur_bal	0.8350	Current balance metrics
revol_util	bc_util	0.8550	Utilization ratios
num_actv_rev_tl	num_rev_tl_bal_gt_0	0.9820	Active revolving trade lines

Raw Financial Variables: DTI and FICO by Outcome



Debt-to-Income Ratio (unfiltered)



FICO Range Low (unfiltered)

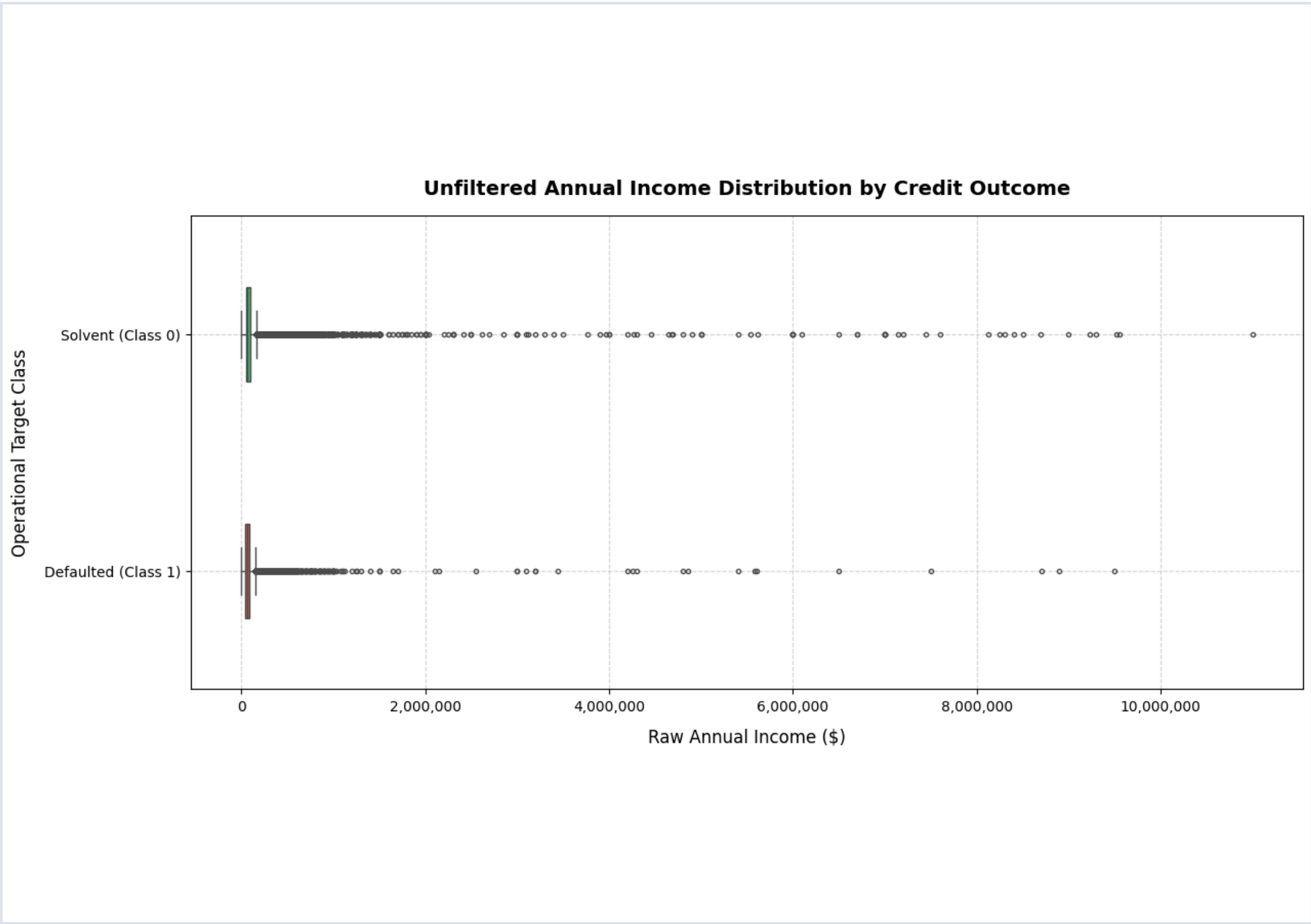
DTI: defaulted borrowers show a visibly higher median debt-to-income ratio (~20% vs. ~17% for solvent borrowers), consistent with intuition — more existing debt relative to income raises repayment strain.

FICO: It stands for **Fair Isaac Corporation**.

It is a widely used credit score ranging from **300 to 850** that summarizes a borrower's credit history and financial behavior, helping lenders assess creditworthiness and predict the likelihood of timely loan repayment.

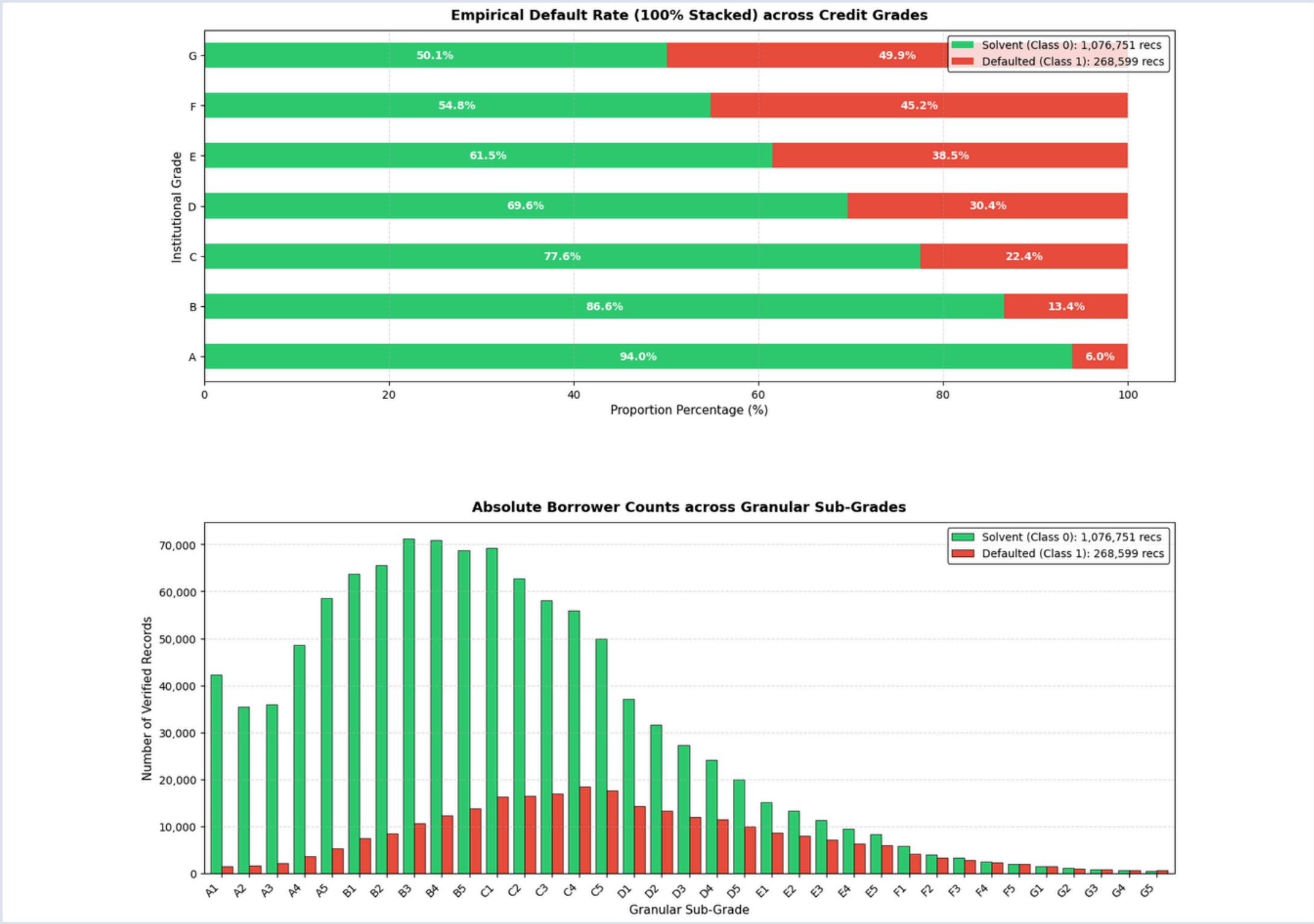
solvent borrowers cluster at higher raw bureau scores; defaulters skew lower, though with heavy overlap — FICO alone is informative but not sufficient, motivating a multivariate model over a bureau-score cutoff.

Annual Income: Extreme Right-Skew Before Treatment



- The unfiltered annual income boxplot reveals a dense band of extreme high-income outliers stretching out to the \$250,000+ cap, dwarfing the interquartile range for both classes.
- Both solvent and defaulted populations show this skew — meaning it reflects genuine heavy-tailed income reporting, not a data-entry error unique to one class.
Downstream implication: this pattern is the direct motivation for the 99th-percentile winsorization step in Data Preparation — capping extreme values so they don't dominate distance-based computations or destabilize tree splits.

Default Rate Climbs Sharply Across Credit Grades



- The stacked bars show a clean monotonic relationship: Grade A loans default at just 6.0%, rising steadily to nearly 50% for Grade G — a textbook validation that LendingClub's own grading system already carries strong, well-calibrated risk signal.
 - The sub-grade volume chart below it shows the portfolio is heavily weighted toward mid-quality loans (B3–C5), with sharply thinning volume in the riskiest tiers (F, G) — a natural long-tail distribution of loan quality.
- Modeling implication:** sub_grade turns out to be one of the single strongest predictive features later in SHAP analysis — this chart is the first hint of why.

SECTION THREE

Data Preparation

Missingness policy, automated multicollinearity pruning, outlier winsorization, MICE imputation, and categorical encoding — turning 152 raw columns into a clean, model-ready matrix.

Missing Data Profile: The Worst Offenders

Column	Missing Count	Missing %
member_id	1,345,350	100.00%
next_pymnt_d	1,345,310	99.997%
orig_projected_additional_accrued_interest	1,341,589	99.72%
hardship_amount	1,339,594	99.57%
hardship_last_payment_amount	1,339,594	99.57%
...	...	...
chargeoff_within_12_mths	56	0.0042%
last_credit_pull_d	55	0.0041%
tax_liens	39	0.0029%
inq_last_6mths	1	0.0001%
zip_code	1	0.0001%

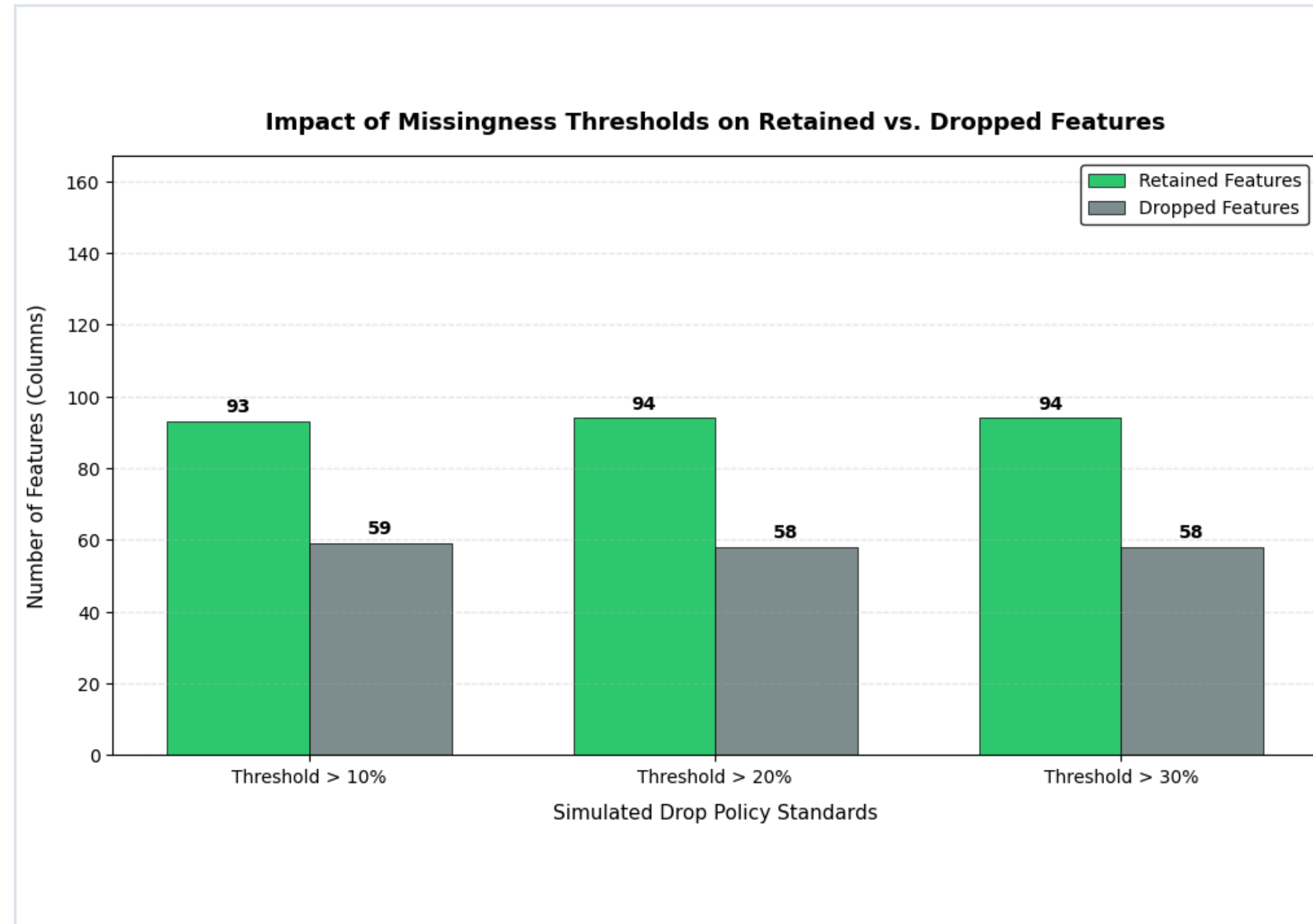
105 of 152 raw columns carry at least some missingness. The profile is bimodal: a cluster of hardship/joint-application fields missing 99%+ of values (structurally absent for most loans), and a long tail of core fields missing under 1%.

Sensitivity Analysis: Threshold Scenario Comparison

Scenario	Columns Dropped	Columns Remaining
Threshold > 10% missing - 134,535 Cells	59	93
Threshold > 20% missing - 269,070 Cells	58	94
Threshold > 30% missing (adopted) - 403,605 cells	58	94

The 20% and 30% scenarios drop the identical set of 58 columns — the missingness distribution has a natural gap between ~20% and ~99%, so the exact cutoff in that range is not a sensitive choice. 30% was adopted as the more conservative, defensible threshold.

Choosing a Missingness Threshold



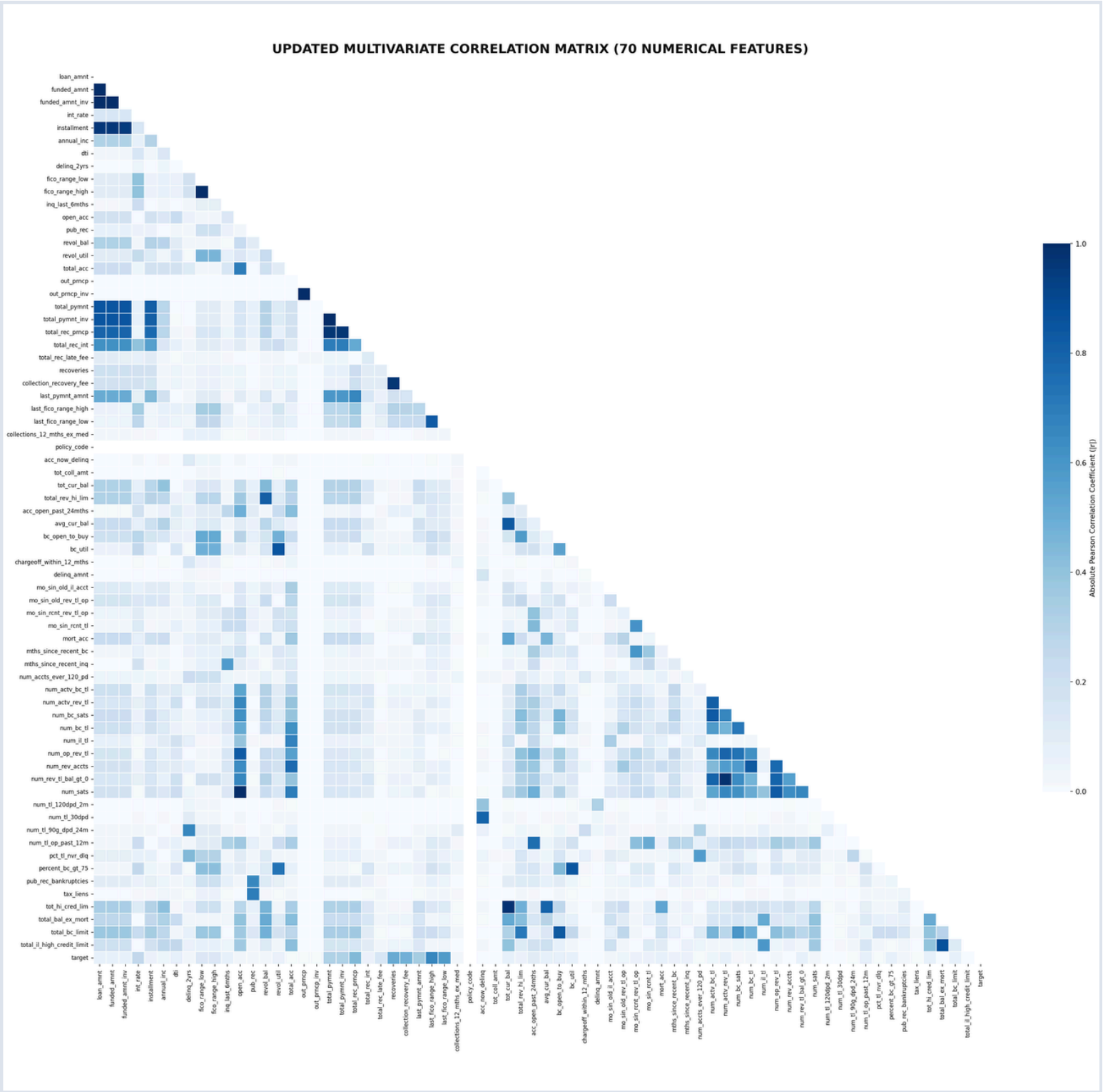
- Every column's missingness rate was profiled, then tested against three candidate thresholds (10% / 20% / 30%) to see how many columns survive each cutoff.
Decision: a 30% missingness threshold was adopted — any column missing more than 30% of its values is dropped outright, before any imputation is attempted.
Rationale: columns with extreme missingness (e.g. hardship or joint-application fields that only populate for a small loan subset) carry too little signal density to justify the imputation risk of fabricating most of their values.
- **Result:** **152** raw columns → **94** columns retained after the threshold cut.

Deduplication & Pipeline Inventory Audit

Stage	Check	Result
Deduplication	Exact duplicate rows scanned across all 94 columns	0 duplicate rows found — dataset already unique per loan
Deduplication	Duplicate loan IDs	0 found — id column is a clean primary key
Inventory Audit	Numeric columns	70 columns
Inventory Audit	Categorical / object columns	24 columns
Inventory Audit	Total feature volume after Phase 3	94 columns × 1,345,350 rows

No duplicate records were found, confirming each row represents a distinct loan — a clean starting point before feature-level pruning.

Re-Auditing Correlation on the Reduced 70-Feature Matrix



- After the missingness cut, the correlation matrix was recomputed on the surviving 70 numeric columns to see which redundant clusters remained.
Finding: the same families persist (loan amount variants, FICO pairs, revolving credit metrics) — confirming that missingness-based pruning and collinearity-based pruning are two independent, complementary filters that both need to run.

Automated Multicollinearity Filtering via Logistic AUC

Feature A	Feature B	r Score	Retained (Higher AUC)	Dropped (Weaker)	Keep AUC	Drop AUC
loan_amnt	funded_amnt	1.000	funded_amnt	loan_amnt	0.5512	0.5511
funded_amnt	funded_amnt_inv	0.999	funded_amnt	funded_amnt_inv	0.5512	0.5510
funded_amnt	installment	0.954	funded_amnt	installment	0.5512	0.5431
PREDICTIVE MULTICOLLINEARITY AUDIT LABS (r > 0.95)						
fico_range_low	fico_range_high	1.000	fico_range_low	fico_range_high	0.5934	0.5934

- Rather than dropping collinear features by hand, every pair with $|r| > 0.95$ was resolved automatically: fit a single-variable logistic regression for each feature in the pair, compare their standalone ROC-AUC, and keep only the stronger predictor.
Example: loan_amnt vs. funded_amnt ($r = 1.000$) — funded_amnt wins narrowly (AUC 0.5512 vs. 0.5511) and is retained.
Example: Out of the features, 11 columns exhibited a strong correlation above 95%. To address this multicollinearity, domain knowledge was applied specifically aligned with the target of predicting defaulters. Consequently, fico_range_low was retained as the primary representative feature, while fico_range_high was removed due to its perfect correlation ($r = 1.000$) with the low range.
- This audit table shows only a representative slice — the full pass evaluated every qualifying pair and took the active feature matrix from 94 to 83 total columns.

Pipeline Evolution: Column Count Through Each Stage

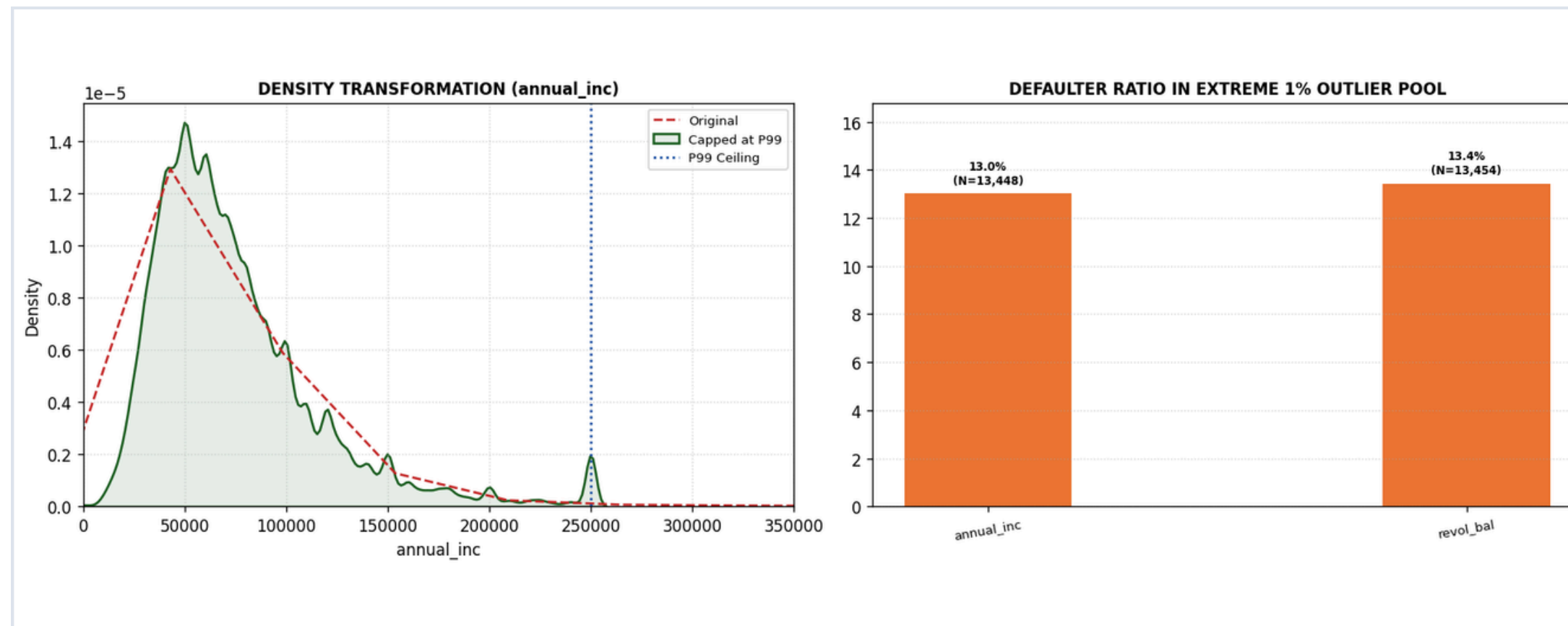
Stage	Columns After	Numeric / Categorical
Raw dataset	152	114/38
Missingness threshold (>30% missing dropped)	94	70 / 24
Automated multicollinearity filter ($ r > 0.95$)	83	59 / 24
Drop residual high-missingness categorical fields (emp_title, emp_length, title, zip_code, last_pymnt_d, last_credit_pull_d)	77	59 / 18
Further categorical / ID cleansing (Additional pruning)	67	58 / 9
One-hot / ordinal / frequency encoding	74	58/16
Leakage purge (8 features) + residual flag cleanup	62 model-ready features	62

Simulated Outlier Shield: Impact Preview on Key Features

Feature	Max Value (Pre-Cap)	P99 Cap Value	Affected Rows	Share of Dataset
annual_inc	\$10,999,200	\$250,000	13,448	1.0%
revol_bal	\$2,904,836	\$94,554	13,454	1.0%

At this pipeline checkpoint (post target-filtering, pre-imputation), `annual_inc`'s max is \$10.99M and `revol_bal`'s is \$2.9M — both far above their 99th-percentile bounds. This is the direct evidence that motivated winsorization rather than simple deletion.

Winsorization: Capping Without Discarding Data



Method: values above the 99th percentile are capped (winsorized) to the P99 value rather than deleted — preserving all 1.34M rows while neutralizing the distortion extreme outliers cause in scaling and tree-split thresholds.

Why cap instead of drop: at this scale, deleting outlier rows would disproportionately remove genuine high-income or high-balance borrowers, introducing selection bias rather than removing noise.

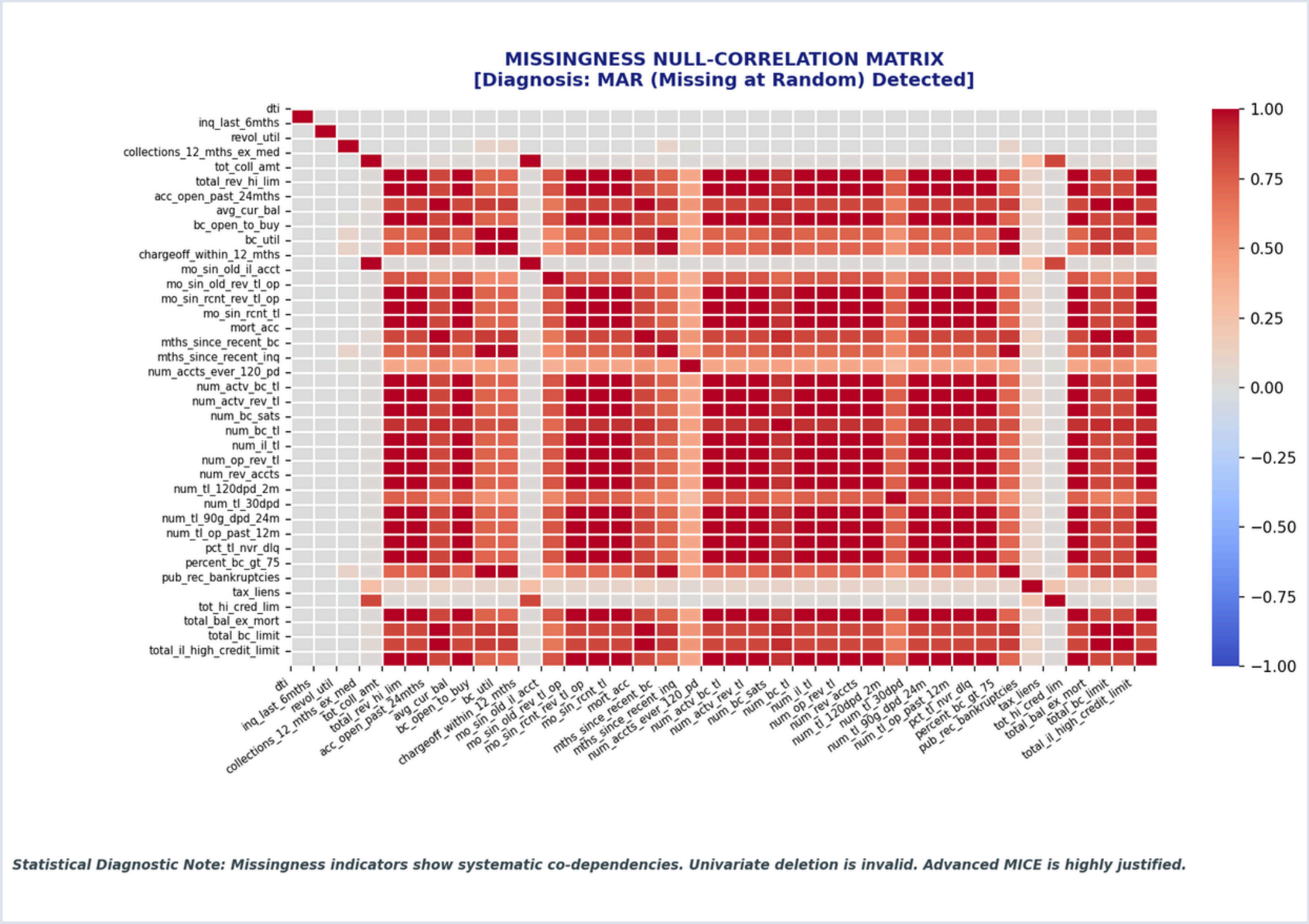
- The comparative density plot confirms defaulter vs. solvent shape is preserved after capping — winsorization changes the tails, not the underlying relationship with the target.

Physical Outlier Transformation Log (Full Column Scan)

Capped Attribute	Max (Pre-Winsor)	P99 Restriction	Max (Post-Winsor)	Status
funded_amnt	40,000.0	35,000.0	35,000.0	CAPPED
int_rate	31.0	26.3	26.3	CAPPED
annual_inc	10,999,200.0	250,000.0	250,000.0	CAPPED
dti	999.0	38.5	38.5	CAPPED
delinq_2yrs	39.0	4.0	4.0	CAPPED
fico_range_low	845.0	800.0	800.0	CAPPED

This is the live execution log applied across every numeric column in order (first 6 of many shown) — distinct from the simulated-impact preview above. dti's raw max of 999% and annual_inc's \$11M outlier are striking confirmations of why the 99th-percentile cap was necessary.

Diagnosing the Missingness Mechanism: MAR



Anticipated Challenge #2: Structured Missing Data

Method: a null-correlation matrix — correlating each column's missingness indicator (1 if missing, 0 if present) against every other column's missingness indicator.

computing their cross-correlation, we evaluate if missingness patterns are systematically coupled (proving MAR dynamics) or entirely decoupled (MCAR).

Diagnosis: Missing At Random (MAR). Strong, systematic co-missingness blocks appear across revolving-credit and account-age metrics — meaning missing patterns are systematically linked to other observed variables in the pipeline, satisfying the MAR assumption..

Why this matters: MAR justifies model-based Multivariate Imputation by Chained Equations (MICE) instead of naive mean/median filling or listwise deletion, both of which would bias the risk features that matter most.

MICE: Multivariate Imputation by Chained Equations

Why not simple imputation: mean/median filling ignores relationships between features and would flatten exactly the credit-risk signal these variables carry.

Method: MICE models each incomplete column as a function of the other columns in an iterative, chained regression process, filling gaps with statistically consistent, relationship-aware estimates.

Engineering constraint: at 1.34M rows × dozens of columns, MICE was executed in chunked batches outside the notebook's interactive runtime to manage memory, then the imputed checkpoint was reloaded for downstream phases.

- Batch Processing: The unified dataframe is mathematically segmented into isolated horizontal blocks of 300,000 records per sub-cycle.
- Variance Protection: MICE utilizing Decision Tree Regressors is applied independently to each chunk, ensuring local covariance preservation while capping memory footprints under 4 GB.
- Re-reconstruction: Imputed batches are sequentially stacked to recover the exact structural integrity of the 83-column space.

MICE Imputation Integrity Report

Post-imputation audit: distributions of key variables (interest rate, DTI, FICO, income) were re-plotted after imputation.

MICE Batch Execution Metric	Value / Log
Total processing blocks (chunked batches)	5 dynamic chunks
Total unresolved numeric missing cells (pre-imputation)	2,172,128 nulls
Total unresolved numeric missing cells (post-imputation)	0 nulls (fully imputed)
Data matrix integrity bounds	100% secure & recovered

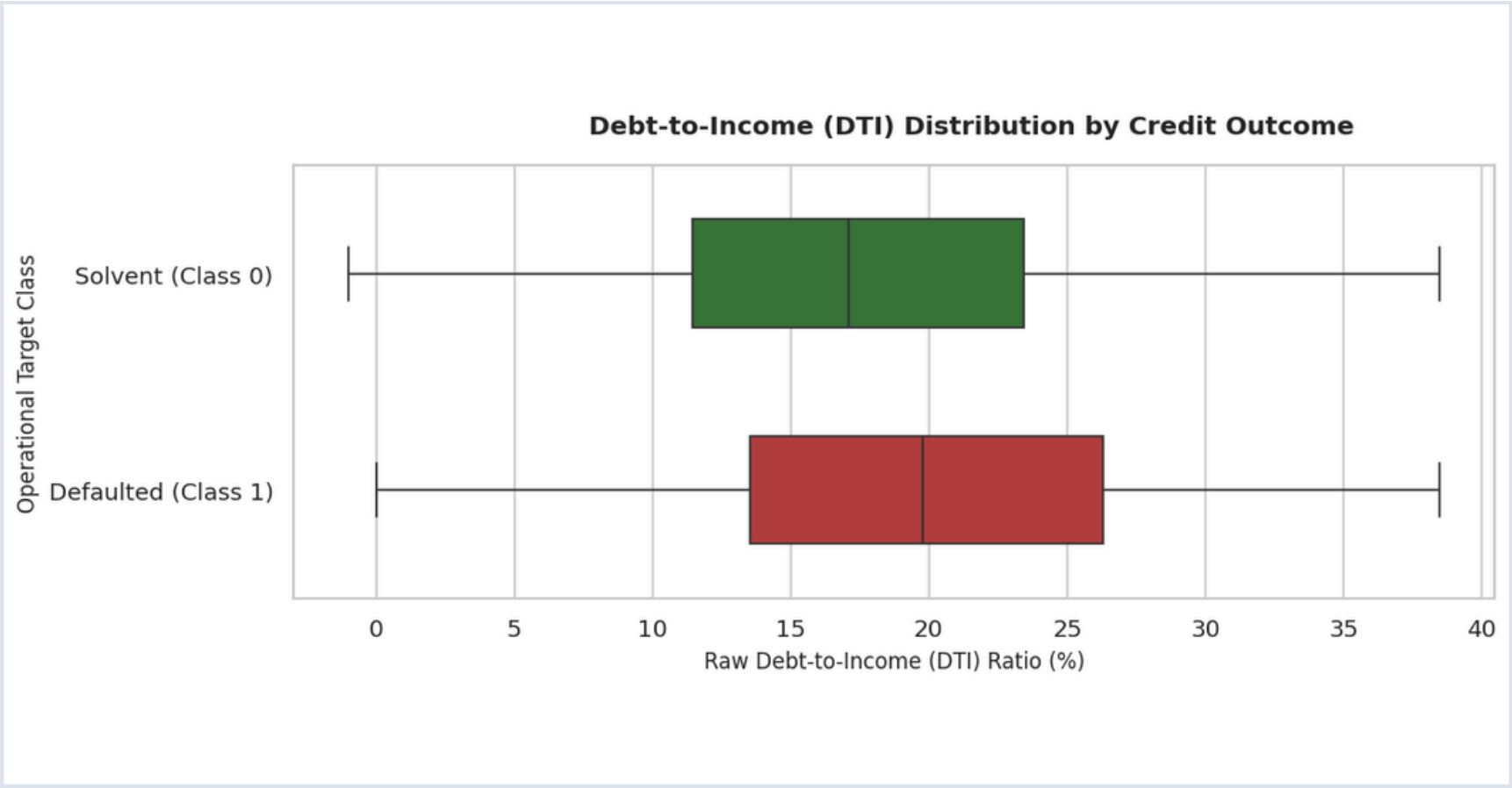
MICE processed the full dataset in 5 memory-optimized chunks of ~269,000 rows each (IterativeImputer with a DecisionTreeRegressor estimator), closing all 2.17 million missing numeric cells without a single residual null.

SYSTEM INTEGRITY AUDIT REPORT

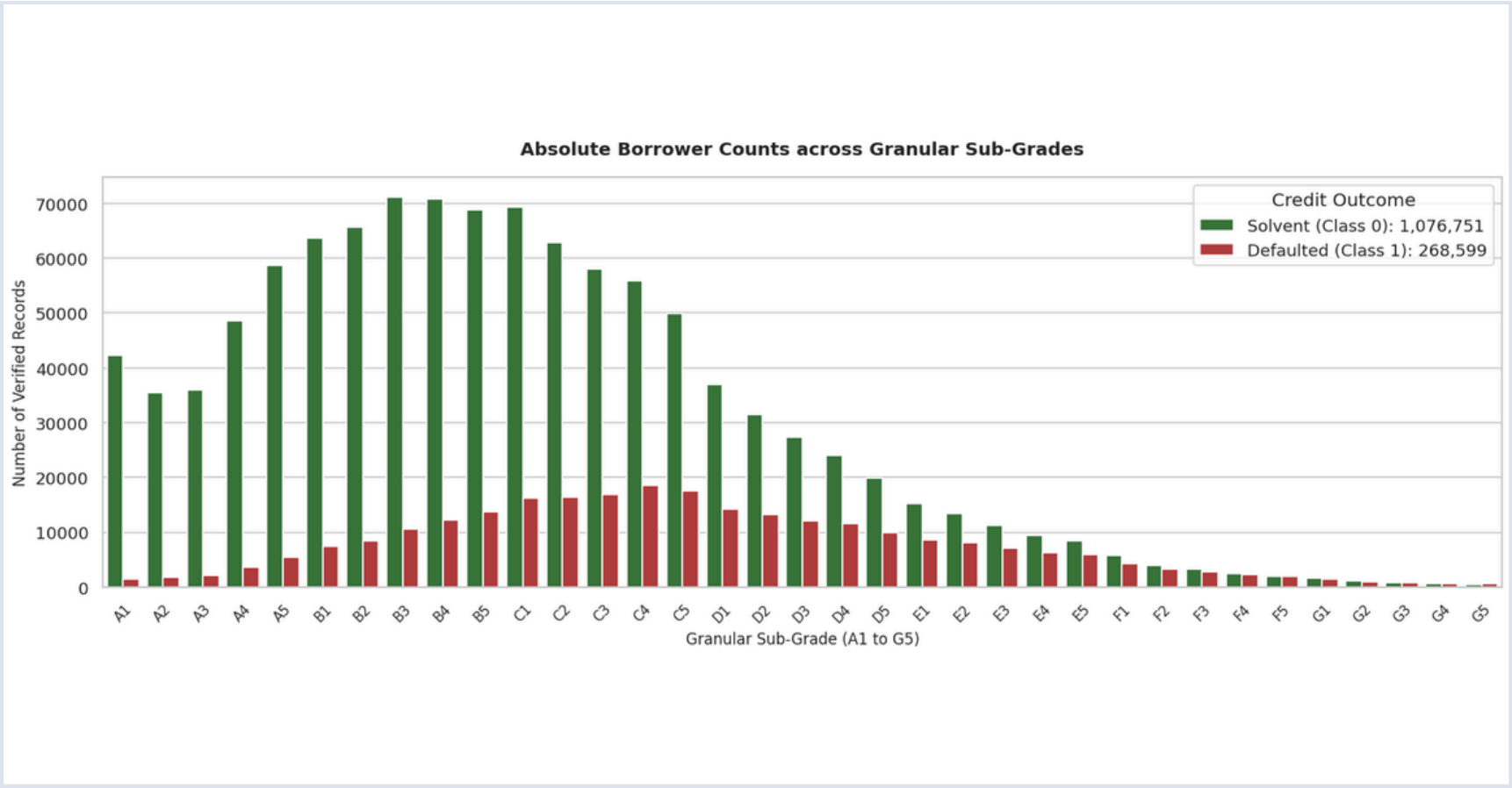
Database Global Structural Metric	Current Stabilized Metric Value	Operational Pipeline Status
Total Verified Observations (Row Volume)	1,345,350	100% Complete & Stable
Absolute Remaining Missing Cells (Nulls)	0	Absolute Zero Bound Achieved
Active Predictor Features (Numerical Pool)	58 Columns	Ready for Global Scale Transformations
Active Predictor Features (Categorical Pool)	9 Columns	Ready for Low-Dimensional Encoding

```
=====
📊 IDENTIFIED CATEGORICAL VARIABLES (9 Columns)
=====
01. sub_grade
02. home_ownership
03. verification_status
04. loan_status
05. purpose
06. addr_state
07. application_type
08. hardship_flag
09. debt_settlement_flag
=====
```

Distributions Remain Stable After Cleaning



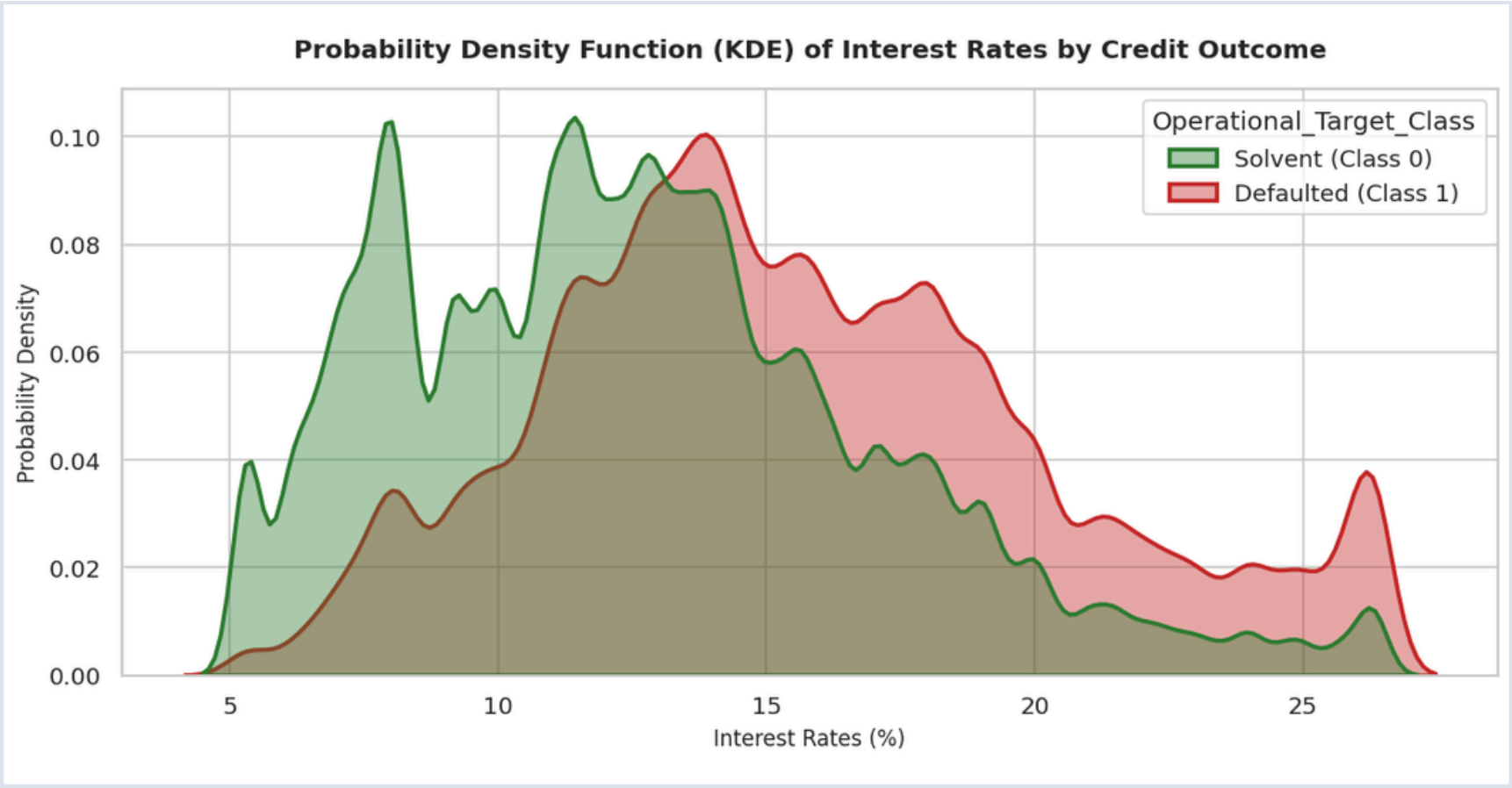
DTI — post-imputation, post-winsorization



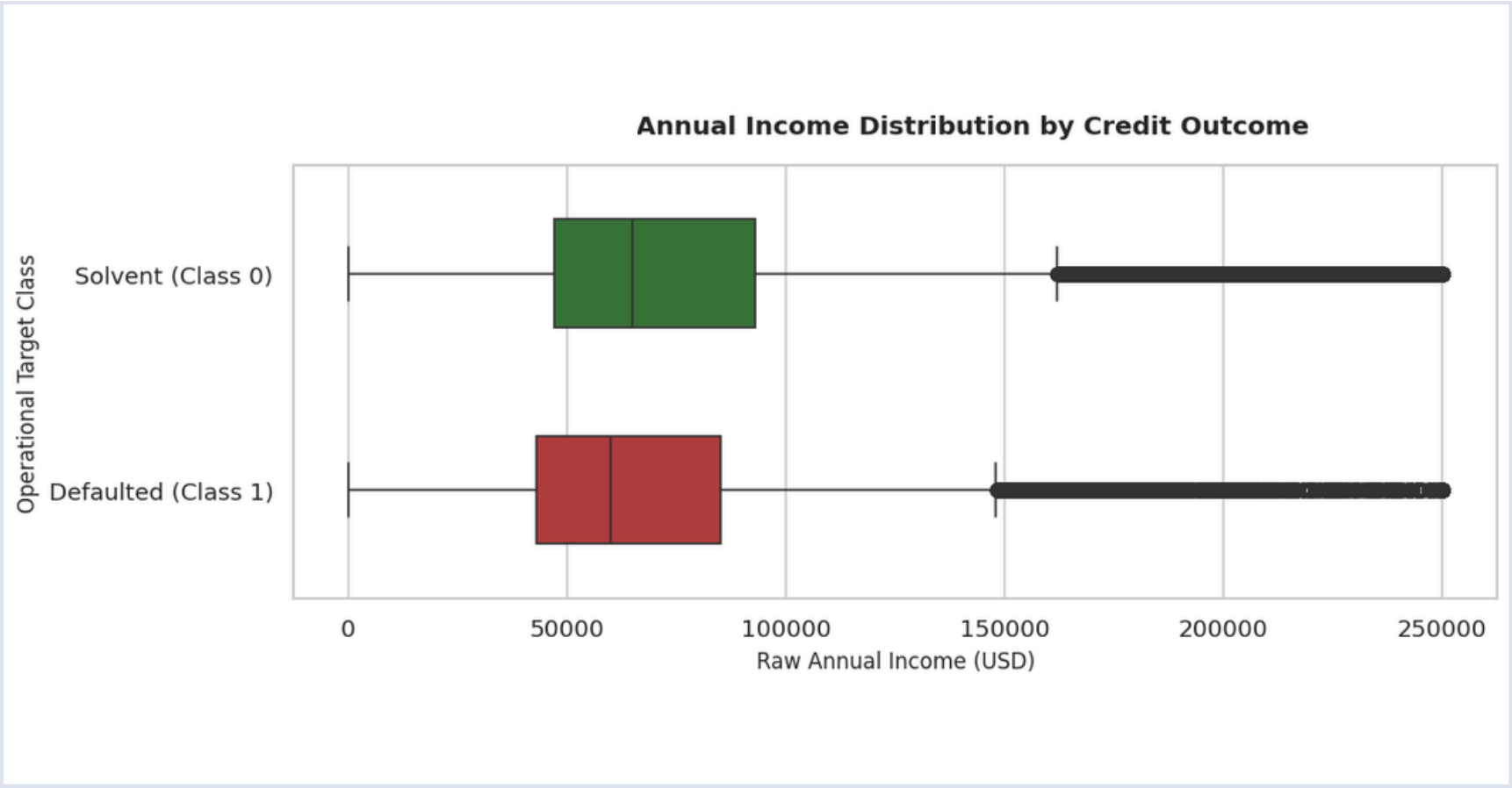
Annual Income — post-winsorization

- These "after" checkpoints re-run the same diagnostic plots from the raw-data stage, now on the cleaned, imputed, winsorized dataset (1,345,350 rows × 83 columns at this checkpoint).
Verification: the DTI gap between solvent and defaulted borrowers is preserved, and the annual income boxplot is now legible and uncompressed — winsorization successfully bounded the extreme right-skew without erasing the class separation.

Two More Checkpoints: Interest Rate and FICO



Interest Rate KDE — post-cleaning

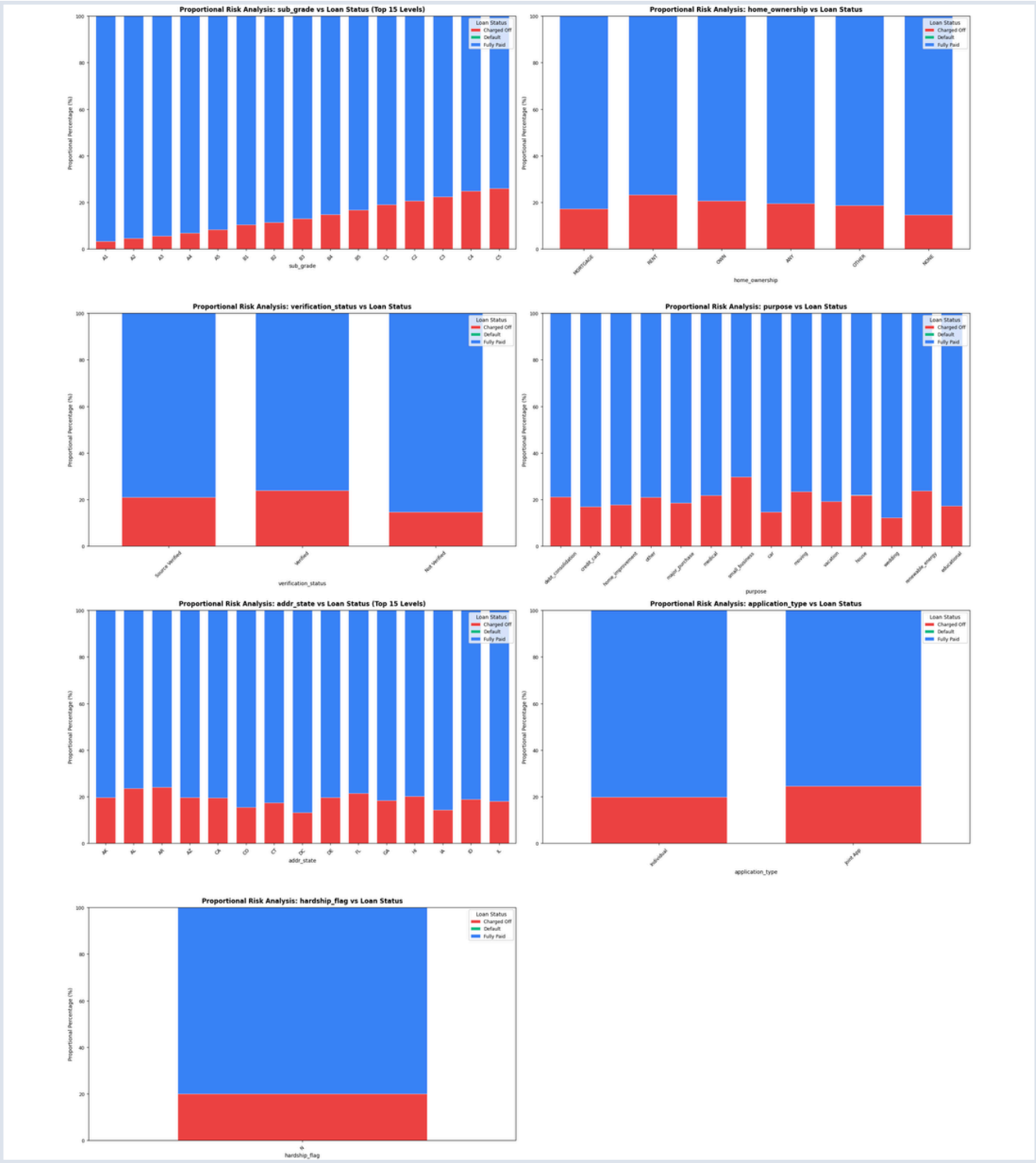


Raw Annual Income

Interest rate: the bimodal solvent-vs-defaulted density shape from the raw-data stage survives cleaning intact — the same separation pattern, just on a fully imputed, outlier-capped matrix.

FICO: the boxplot shape and class separation are unchanged post-cleaning, confirming MICE imputation did not artificially compress or shift the bureau-score distribution.

Categorical Feature Landscape vs. Default Outcome



sub_grade: clean monotonic risk gradient from A1 (lowest default) to G5 (highest) — the strongest categorical signal in the dataset.

home_ownership: renters default modestly more than mortgage holders or owners.

purpose: small business loans show elevated default risk; renewable energy and vacation loans show relatively lower risk.

verification_status: counter-intuitively, "Verified" applicants show a slightly higher default rate than "Not Verified" — a nuance worth flagging rather than over-interpreting causally.

application_type: joint applications default somewhat more often than individual applications.

Encoding Strategy by Variable Type

Feature	Encoding Method	Reason
sub_grade	Ordinal Encoding	The categories have a natural ranking (A1 to G5), making ordinal encoding the most appropriate choice.
home_ownership	One-Hot Encoding	This feature contains only a few nominal categories without any inherent order.
verification_status	One-Hot Encoding	The categories are nominal and have no meaningful ranking.
application_type	Binary Encoding	This feature contains only two categories, making binary encoding simple and efficient.
purpose	Frequency Encoding	This feature has relatively high cardinality. Frequency encoding preserves useful information while preventing a large increase in the number of features.
addr_state	Frequency Encoding	Since this feature contains many unique states, frequency encoding effectively reduces dimensionality compared to one-hot encoding.
hardship_flag	Removed	The feature provides little useful information due to its extremely low variance.
debt_settlement_flag	Removed	This feature may introduce data leakage because it is generated after the loan outcome is known.

Encoding method and exact mapping are read directly from the notebook's own transformation log (e.g. "application_type mapped to {'Individual': 0, 'Joint App': 1}").

Class Imbalance Management Strategy

Class Imbalance Challenge

Scenario: Target exhibits 19.96% default distribution (~4:1 solvent-to-default ratio).

Risk: Standard training risks underestimating defaults, causing critical Type II errors(Fals negative).

Algorithmic Weighting Framework

Strategy: Replaced memory-heavy oversampling (e.g., SMOTE) with algorithmic class weighting.

Efficiency: Avoids computational lag and synthetic database inflation on 1.3M+ rows.

Implementation: Exact scale ratios passed directly to model engines (scale_pos_weight in XGBoost/LightGBM; class_weight='balanced' in Logistic Regression).

Impact: Optimizes the loss function to heavily penalize False Negatives without altering original data structure.

Class Imbalance Management Strategy

```
=== Phase 4.3.0: Calculating Algorithmic Class Imbalance Weighting Factors ===
```

```
=====
📊 ALGORITHMIC IMBALANCE WEIGHTING PARAMETERS COMPUTED SUCCESSFULLY
=====
```

```
-> Total Non-Default Training Observations (Class 0) : 861,401
```

```
-> Total Default Training Observations      (Class 1) : 214,879
```

```
-----
-> Calculated XGBoost/LightGBM 'scale_pos_weight'      : 4.0088
```

```
-> Configured Scikit-Learn 'class_weight'              : balanced
```

```
=====
Pipeline Notice: Imbalance metrics are calculated and ready to be fed into objective functions.
```

SECTION FOUR — THE CENTRAL STORY OF THIS PROJECT

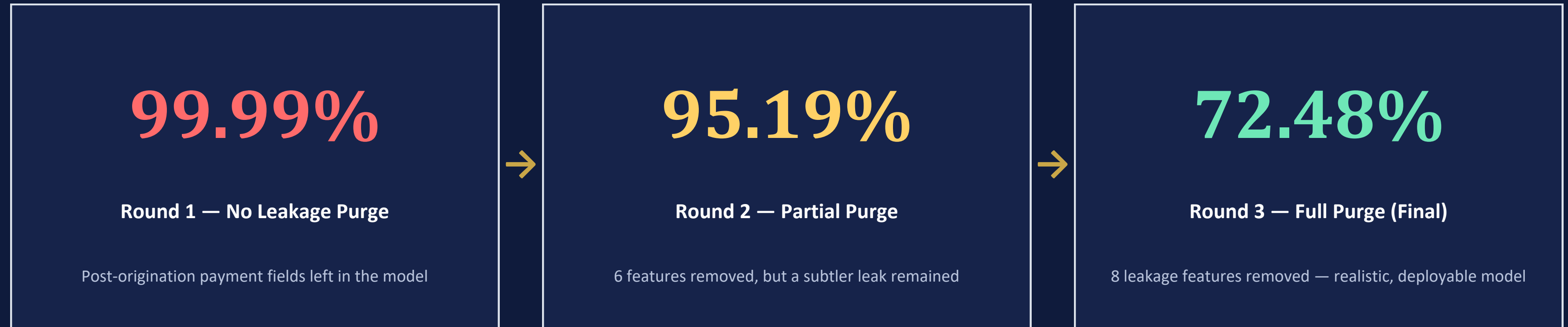
The Leakage Investigation

Twice, this pipeline produced a model that looked almost impossibly good. Both times, that was a red flag rather than a result — and both times, forensic SHAP analysis found exactly why. This is the audit trail.

"Too Good to Be True" Is a Diagnosis, Not a Result

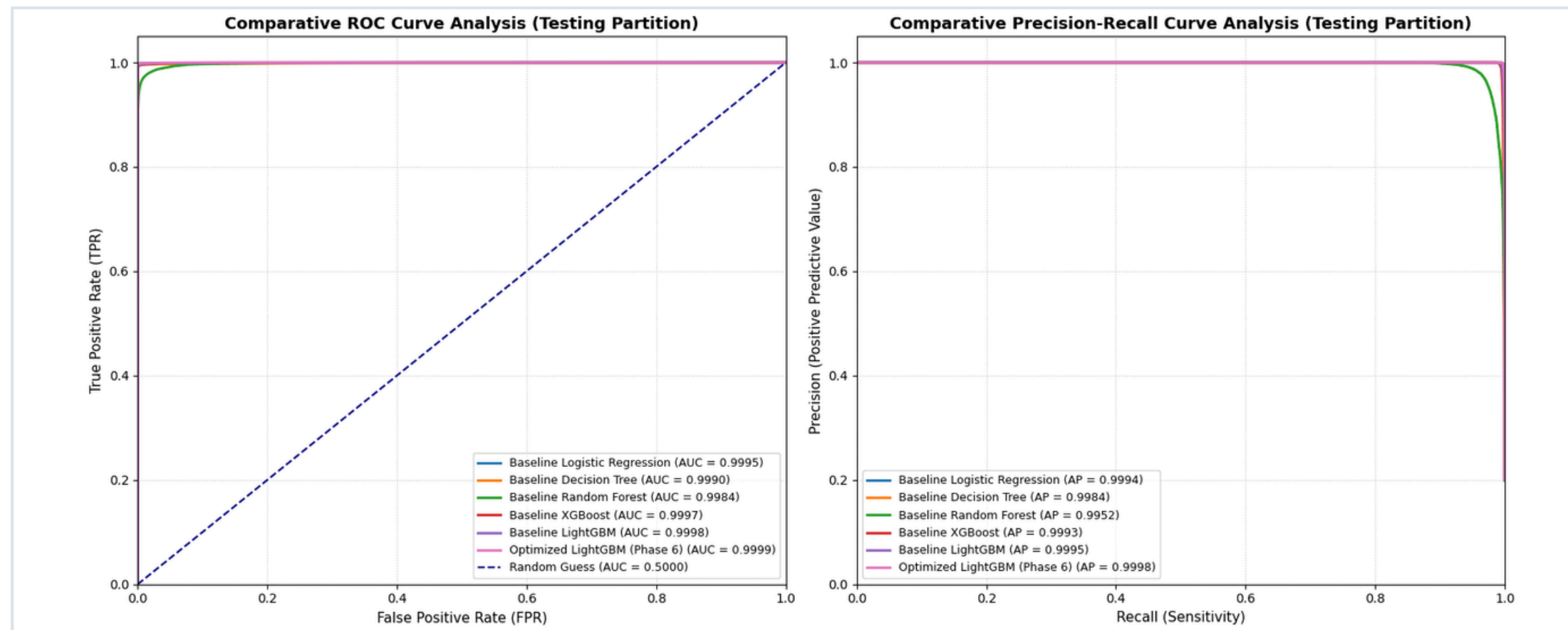
- Data leakage happens when information that would not be available at prediction time sneaks into the training features — the model isn't learning to predict the future, it's learning to read an answer that's already been written into the data.
The danger: a leaky model reports spectacular offline metrics (ROC-AUC near 1.0) and then fails catastrophically the moment it faces genuinely unknown, real-world applicants.
- This project went through three iterations of the same modeling pipeline. We are presenting all three — not just the final clean one — because the investigation itself is the strongest evidence that the final 72.48% ROC-AUC is real, earned, and trustworthy.

Three Rounds, Three Very Different Answers



- Test-set ROC-AUC, Optimized LightGBM, at each stage of the investigation. Every other modeling choice (train/test split, hyperparameter search space, class weighting) was held identical across all three rounds — the only variable changing is which features were allowed into the model.

No Purge: A Model That Is Too Good to Trust



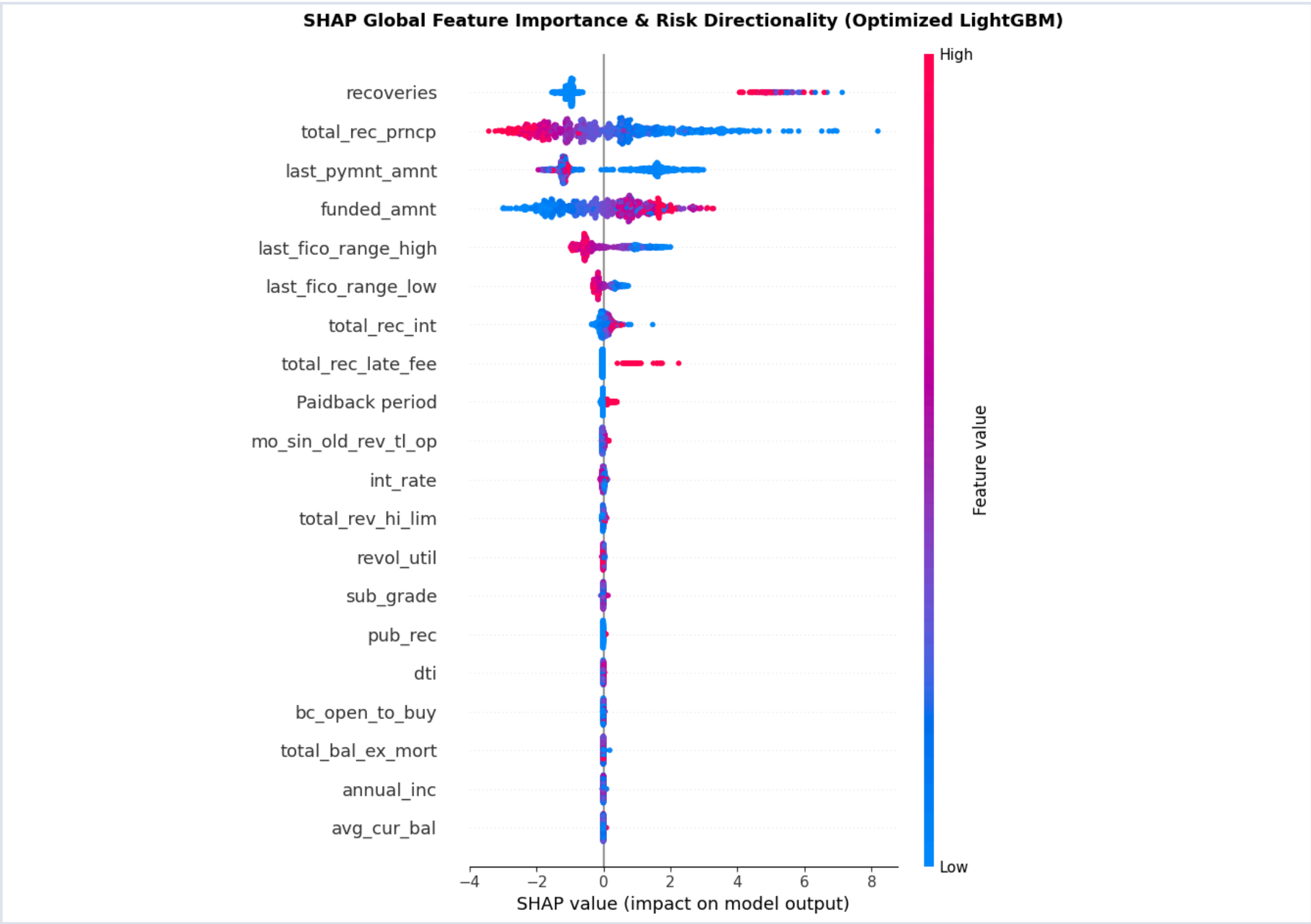
Round 1: Raw Feature Set, Zero Purge

Result: every model — even plain Logistic Regression — hits ROC-AUC above 0.998. The Optimized LightGBM reaches 0.9999, essentially a perfect classifier.

Why this is alarming, not exciting: real-world credit default is genuinely hard to predict from application-time information; near-perfect separation is the signature of leakage, not skill.

- At this stage, the full raw feature set was used with no leakage screening — including several fields that LendingClub only populates after a loan begins repayment.

SHAP Reveals the Culprits: Post-Originaton Payment Fields



recoveries and **total_rec_prncp** tower over every other feature — their SHAP impact is 3–5× larger than the next-most-important variable, an unmistakable outlier signature.

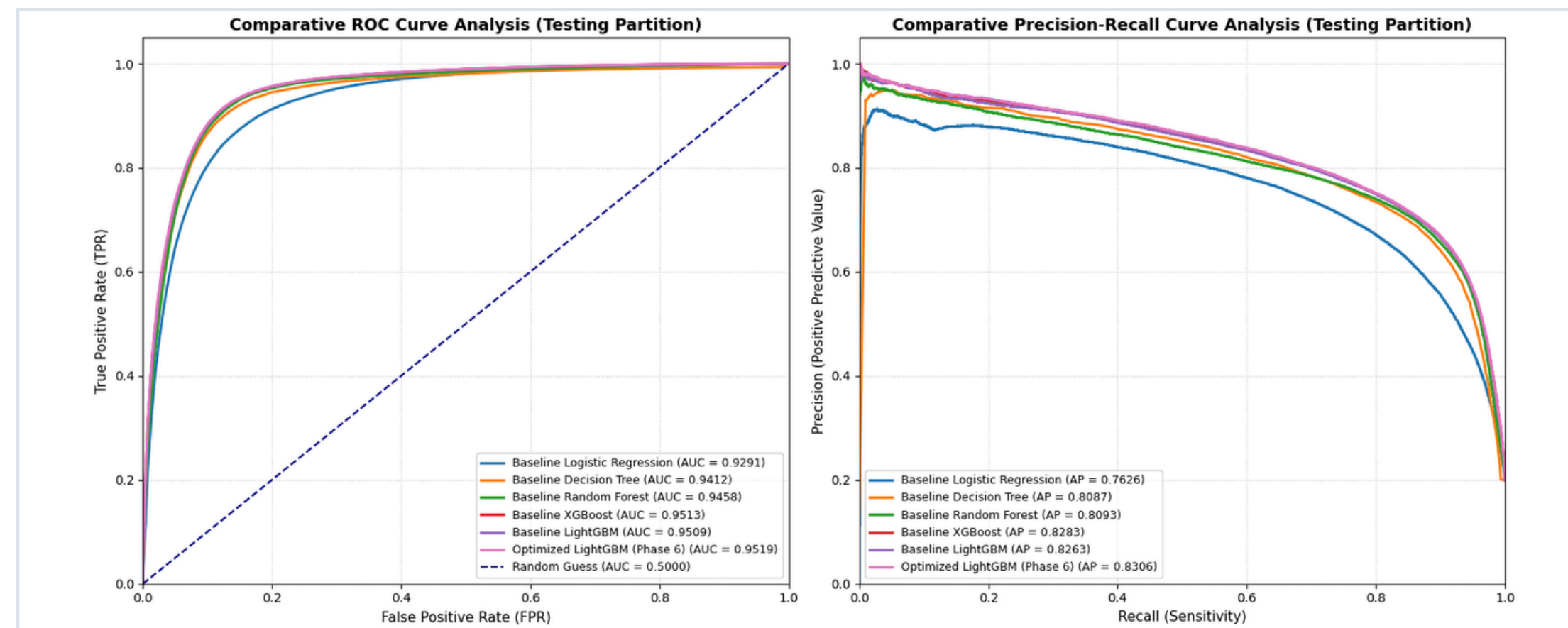
- Both fields only exist after a loan has already gone through collections or been substantially repaid — they are direct byproducts of the outcome, not predictors of it.
- Fix identified:** purge all fields generated during loan servicing rather than at origination — recoveries, total_rec_prncp, collection_recovery_fee, last_pymnt_amnt, total_rec_int, total_rec_late_fee (6 columns).

Every Model Is Suspiciously Perfect

Model	Test ROC-AUC	Test Avg. Precision
Logistic Regression	99.9548%	99.9382%
Decision Tree	99.9009%	99.8417%
Random Forest	99.8378%	99.5193%
XGBoost	99.9713%	99.9318%
LightGBM (baseline)	99.9827%	99.9531%
Optimized LightGBM	99.9918%	99.9794%

Even the weakest model here (Random Forest) still clears 99.8% ROC-AUC — when every architecture, including a plain linear model, scores this close to perfect, the problem is the data, not the modeling.

First Purge Applied — Still Too Good



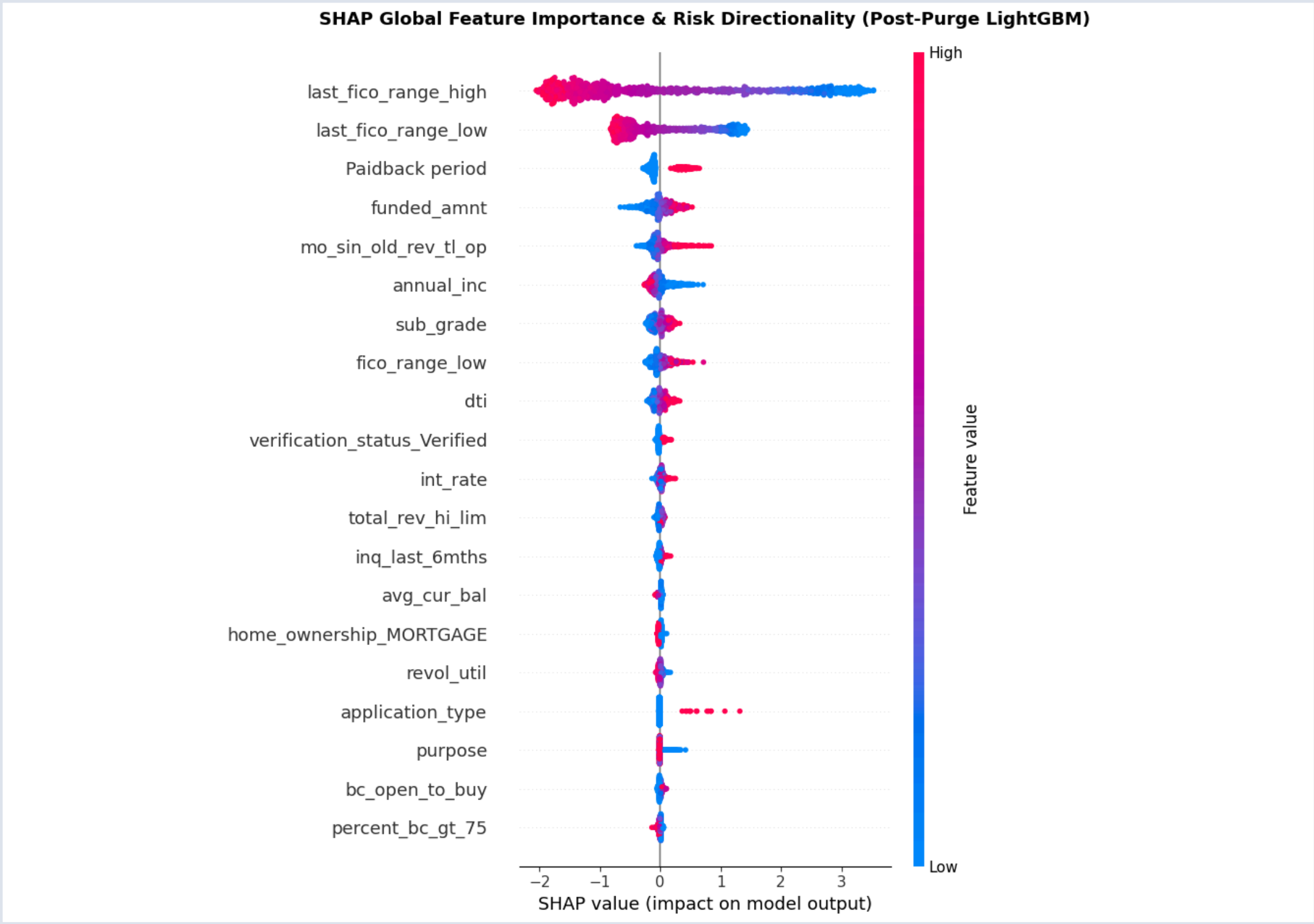
Round 2: 6 Fields Purged, Gap Still Suspicious

Action: the 6 post-origination payment/collection fields identified in Round 1 were removed and the full pipeline was re-run end-to-end.

Result: ROC-AUC dropped meaningfully — from 0.9999 to 0.9519 for the Optimized LightGBM — a real improvement, but still implausibly high for a credit-risk model at application time.

Why we didn't stop here: 0.95 ROC-AUC is still far above what published academic and industry benchmarks report for LendingClub-style default models (typically 0.68–0.75) — the residual gap demanded a second forensic pass.

A Subtler Leak: Retrospective FICO Fields



This is exactly the leak the assignment brief calls a "non-trivial challenge"

`last_fico_range_high` and `last_fico_range_low` now dominate the SHAP ranking, together carrying more explanatory weight than every other feature combined.

The subtlety: these look like ordinary credit-bureau fields, but LendingClub updates them continuously through the life of the loan — the "last" reported FICO score is often captured near or at the point of charge-off, silently encoding the outcome.

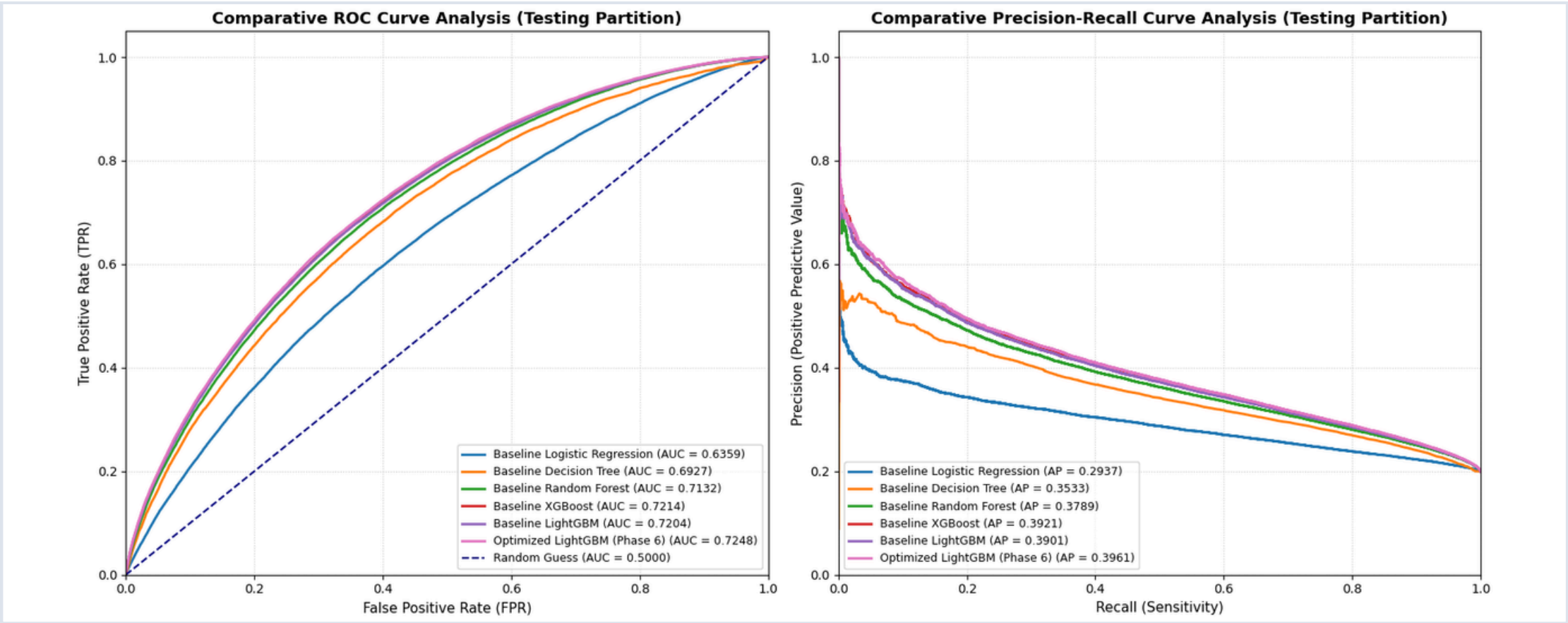
Fix identified: remove `last_fico_range_high` and `last_fico_range_low`, keeping only the origination-time `fico_range_low` / `fico_range_high` that were known when the loan was underwritten.

Still Implausibly High Across Every Model

Model	Test ROC-AUC	Test Avg. Precision
Logistic Regression	92.9070%	76.2616%
Decision Tree	94.1164%	80.8702%
Random Forest	94.5821%	80.9303%
XGBoost	95.1263%	82.8311%
LightGBM (baseline)	95.0947%	82.6346%
Optimized LightGBM	95.1851%	83.0620%

A real improvement from Round 1 (roughly 5–7 points lower ROC-AUC across the board), but still far above the 0.68–0.75 range typical of published LendingClub default models — the residual gap that motivated the second forensic pass.

Ultimate Purge: A Realistic, Deployable Model



Round 3: 8 Fields Purged — Adopted as Final Model

Action: all 8 leakage columns removed in total — the 6 post-origination payment fields plus the 2 retrospective FICO fields — leaving only variables genuinely known at the moment of loan origination.

Result: Optimized LightGBM settles at ROC-AUC = 0.7248, Average Precision = 0.3961 — a large drop from Round 2, and exactly the expected range for a real credit-default model.

Why this is the right answer, not a worse one: a model this size gap lower is a model that will actually generalize to new applicants, because it was never allowed to see their future.

The Full Audit Trail

Round	Features Removed	Test ROC-AUC	Test AP	Verdict
1 — No Purge	None	0.9999	0.9998	near-perfect score signals
2 — Partial Purge	6 (recoveries, total_rec_prncp, collection_recovery_fee, last_pymnt_amnt, total_rec_int, total_rec_late_fee)	0.9519	0.8306	Rejected — still above plausible real-world range
3 — Full Purge (Final)	8 total (+ last_fico_range_high, last_fico_range_low)	0.7248	0.3961	consistent with published

Every round used the identical Optimized LightGBM architecture, train/test split, and class-weighting strategy — the only variable was which columns were allowed into the feature matrix.

SECTION FIVE

Model Building & Hyperparameter Tuning

With a leakage-free, 62-feature matrix in hand, six model configurations were trained and one was tuned — all on an 80/20 stratified split of 1.34M loans.

Stratified Partitioning & Class-Imbalance Handling

Setting	Value
Split method	train_test_split, stratify = target, test_size = 0.20, random_state = 42
Training set	1,076,280 rows × 62 features
Test set (held out)	269,070 rows × 62 features
Class balance handling	Cost-sensitive weighting — no synthetic oversampling
scale_pos_weight (XGBoost / LightGBM)	4.0088 (negative-class count ÷ positive-class count)
class_weight (Logistic Regression, trees)	'balanced'

Stratification guarantees the 80.04% / 19.96% class ratio is preserved identically in both the training and test partitions — essential for a fair evaluation under imbalance.

Six Models, One Consistent Evaluation Protocol

Logistic Regression — L2 penalty, C=1.0, SAGA solver, class_weight='balanced'. The interpretable linear baseline.

Decision Tree — max_depth=12, min_samples_split=50, class_weight='balanced'. Captures non-linear splits, prone to variance.

Random Forest — 15 bootstrapped trees, max_depth=12, max_features='sqrt'. Variance reduction via bagging.

XGBoost — tree_method='hist' for histogram-based binning, scale_pos_weight=4.0088. Fast, regularized boosting.

LightGBM (baseline) — n_estimators=100, max_depth=6, learning_rate=0.05. Leaf-wise growth for speed at scale.

Optimized LightGBM (champion) — same family, tuned via RandomizedSearchCV — see next slide.

RandomizedSearchCV: Why Not Grid Search

The constraint: 1,076,280 training rows × 62 features makes an exhaustive GridSearchCV computationally infeasible — the combinatorial explosion risks out-of-memory failures and unbounded runtime.

The solution: RandomizedSearchCV samples 10 candidate parameter combinations (n_iter=10) from a targeted distribution, validated with 3-fold StratifiedKFold — 30 total model fits, scored on ROC-AUC.

Search space: n_estimators {50,100,150} · learning_rate {0.03,0.05,0.1} · max_depth {4,6,8} · num_leaves {20,31,45} · subsample {0.7,0.8,1.0} · colsample_bytree {0.7,0.8,1.0}.

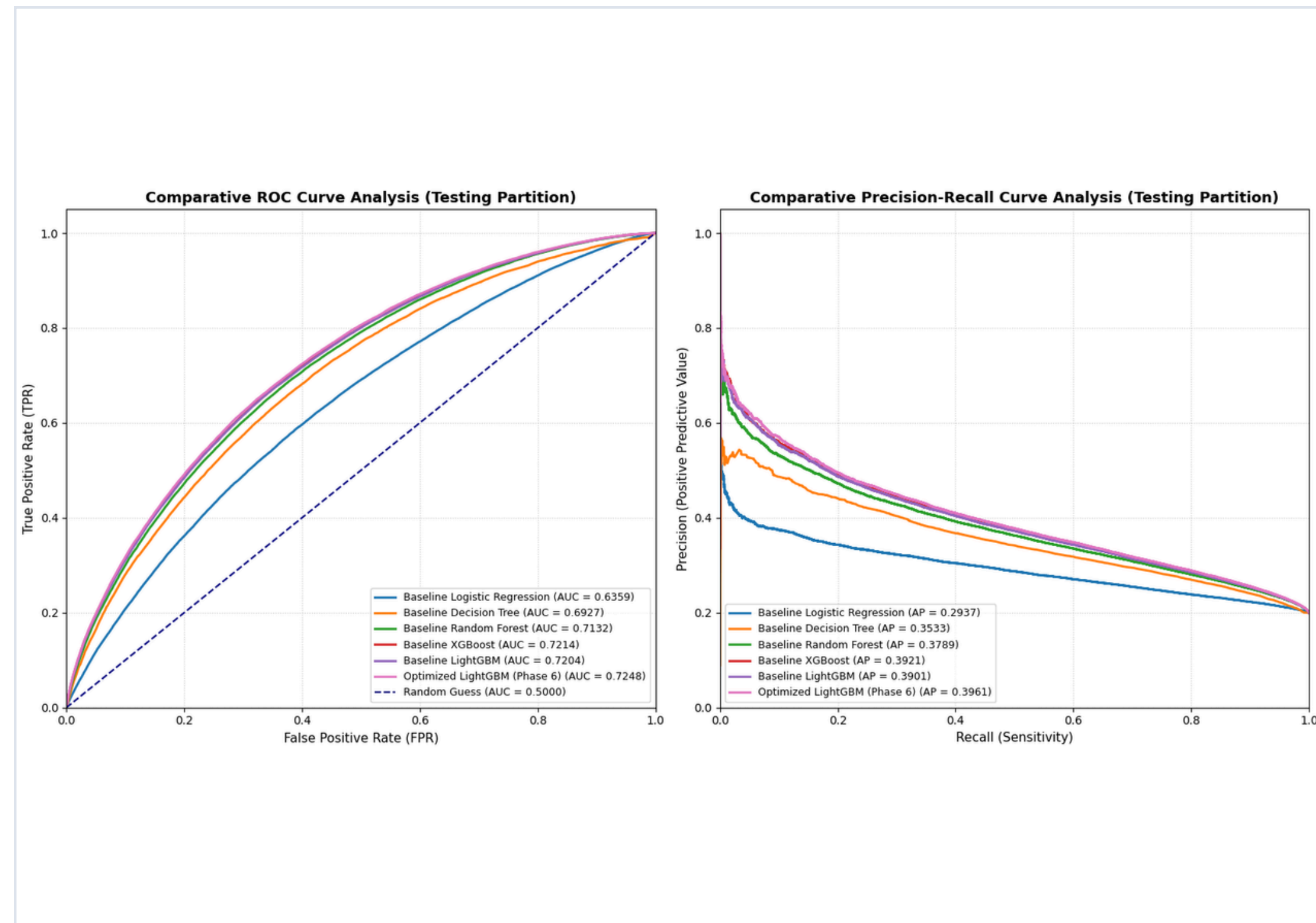
Winning configuration: n_estimators=100, learning_rate=0.1, max_depth=8, num_leaves=20, subsample=0.7, colsample_bytree=0.8 — wide-but-shallow trees with dual bagging, chosen for anti-overfitting on 1M+ rows.

SECTION SIX

Model Evaluation

All six candidates are benchmarked on the same 269,070-row held-out test set, using ROC-AUC and Average Precision — the right metrics for an imbalanced, ranking-driven credit decision.

ROC and Precision-Recall Curves — Final, Leakage-Free Models



- All six models sit clearly above the random-guess diagonal, confirming genuine (if modest) predictive signal survives the leakage purge.
Ranking: Optimized LightGBM (0.7248) edges out XGBoost (0.7214) and baseline LightGBM (0.7204), with Random Forest (0.7132), Decision Tree (0.6927), and Logistic Regression (0.6359) trailing.
Why Average Precision matters here: with only ~20% positive class, AP (area under the Precision-Recall curve) is a more honest summary of minority-class performance than ROC-AUC alone — the champion's AP of 0.3961 means roughly 2× better than a random classifier at the same recall level.

Final Leaderboard — Held-Out Test Set (269,070 loans)

Model	Test ROC-AUC	Test Avg. Precision	Notes
Logistic Regression	0.6359	0.2937	Linear baseline — underfits non-linear risk interactions
Decision Tree	0.6927	0.3533	Single tree — moderate lift, higher variance
Random Forest	0.7132	0.3789	Bagging improves stability over a single tree
XGBoost	0.7214	0.3921	Strong boosting baseline, close second
LightGBM (baseline)	0.7204	0.3901	Comparable to XGBoost, faster to train
Optimized LightGBM	0.7248	0.3961	Champion — selected for tuning + interpretability stack

Optimized LightGBM is adopted as the champion: best ROC-AUC and AP, plus native SHAP support for the explainability requirements of a credit-decisioning use case.

Generalization Stability: Train vs. Test Mean Risk Probability

Model	Mean Train Prob.	Mean Test Prob.	Stability Delta
Logistic Regression	48.1954%	48.1860%	0.0094%
Decision Tree (depth 12)	44.8002%	44.8084%	0.0082%
Random Forest (15 trees)	45.1393%	45.1145%	0.0248%
LightGBM (baseline)	45.5272%	45.5048%	0.0224%
XGBoost (baseline)	45.6167%	45.5929%	0.0238%

Beyond ROC-AUC/AP, each model's mean predicted probability was compared between training and test partitions. All five models show a stability delta under 0.03 percentage points — strong evidence that none of the champion candidates are overfitting the training data.

Why the Champion Model Was Chosen

Marginal but consistent gains: boosting methods (XGBoost, LightGBM) outperform bagging (Random Forest) and single trees, which in turn outperform the linear baseline — the expected ordering for tabular credit data with non-linear feature interactions.

Tuning added real value: hyperparameter search lifted LightGBM from 0.7204 to 0.7248 ROC-AUC — a modest but genuine improvement, not the artificial jump seen when leakage was present.

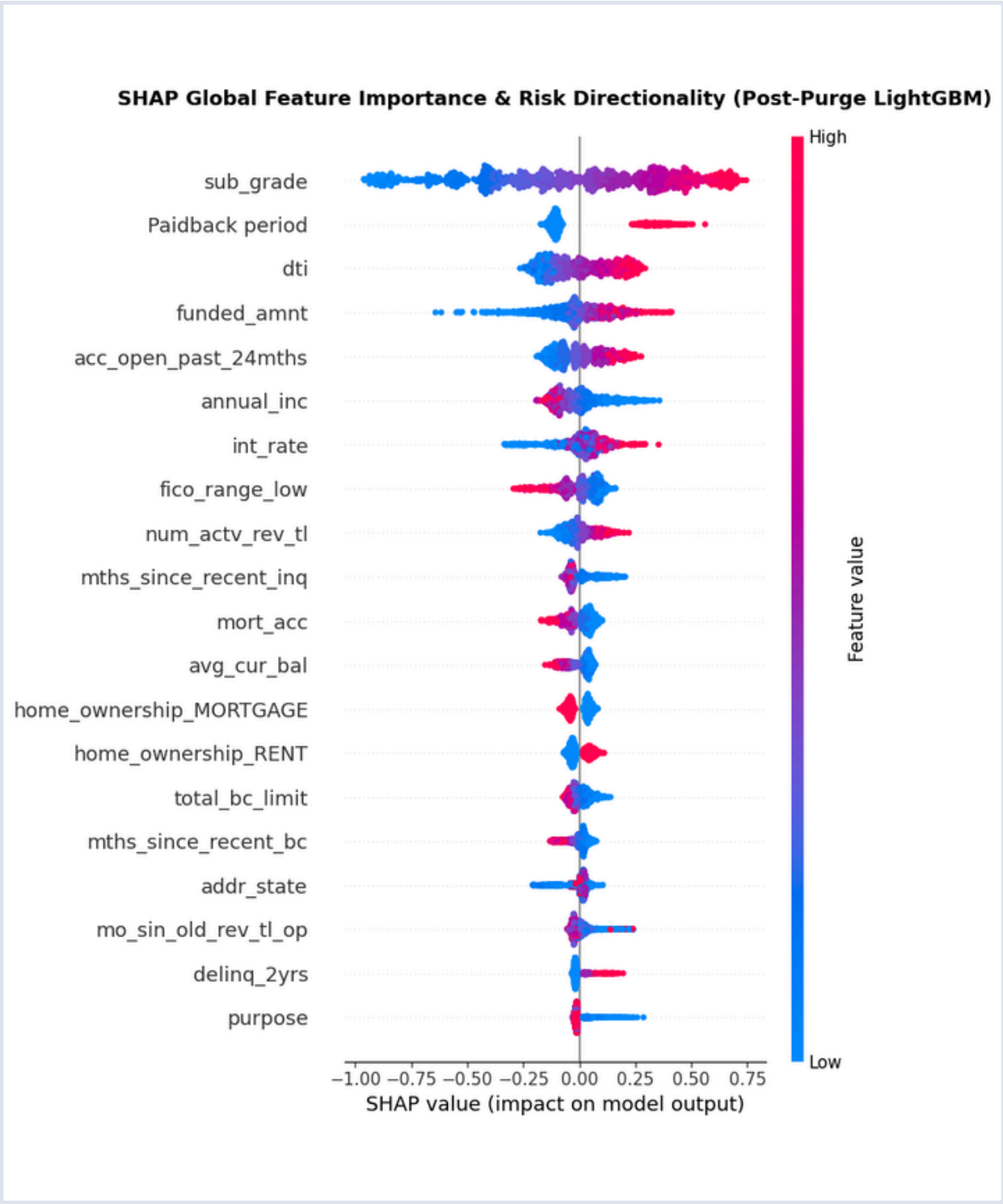
Operational fit: LightGBM's native SHAP TreeExplainer support, fast inference, and leaf-wise efficiency at 1M+ rows make it the practical choice for a production credit-scoring pipeline, not just the top scorer on paper.

SECTION SEVEN

Interpretability & Calibration

A credit decision needs to be explainable, not just accurate. SHAP opens the model's reasoning at both the population and individual-applicant level; calibration checks whether its probabilities can be trusted at face value.

SHAP Confirms the Leakage Purge Worked



sub_grade is now the single strongest driver — exactly the legitimate, origination-time signal we'd expect, not a post-outcome artifact.

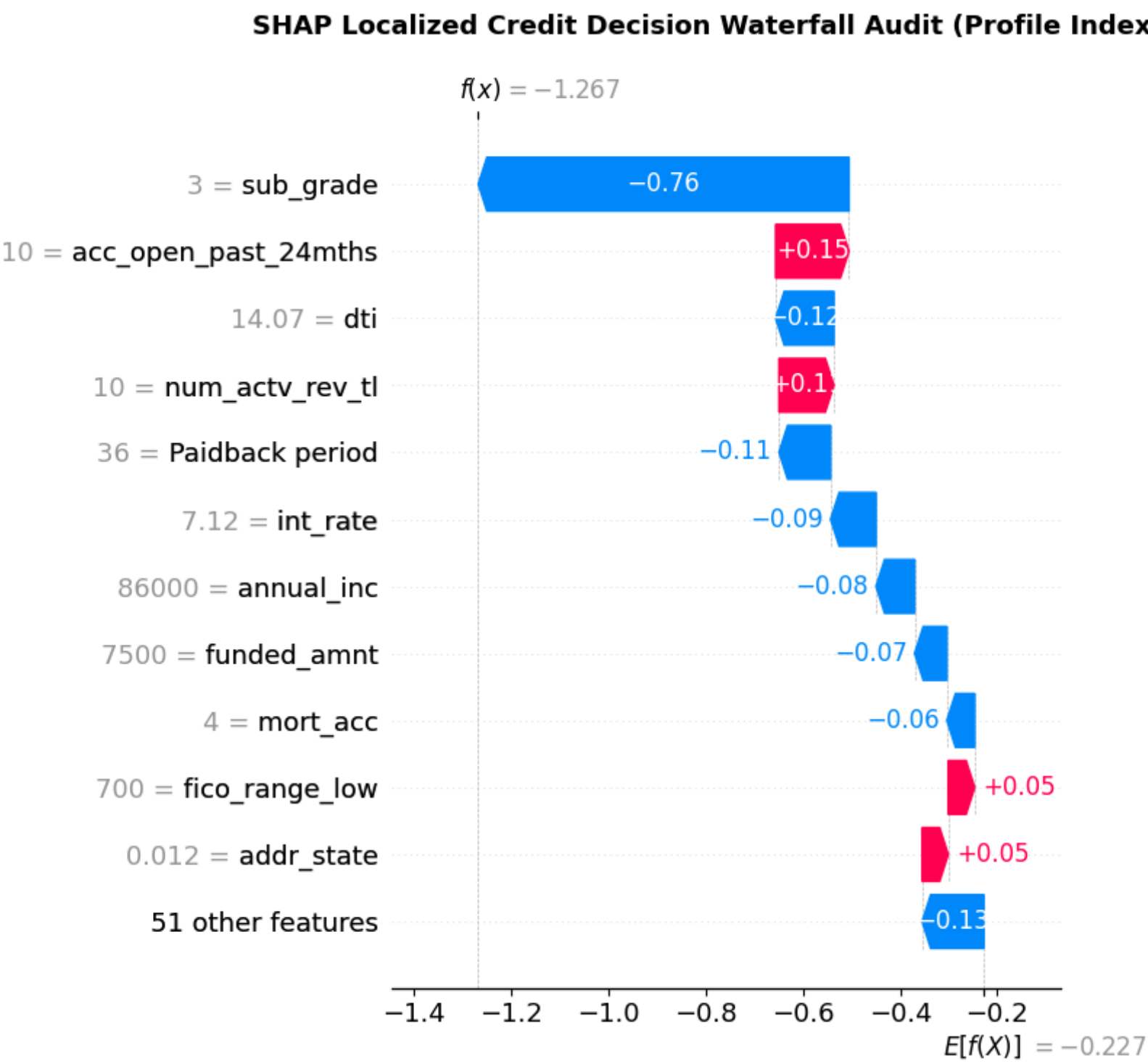
Paidback period (loan term), dti, funded_amnt, and acc_open_past_24mths round out the top five — all genuinely known before a loan is disbursed.

This chart is the payoff of Section 4: compare it to the Round 1 and Round 2 SHAP plots — no single feature dominates unnaturally, and every top driver is something a real underwriter could justify using.

SHAP Feature Ranking: Top 20 of 62 Model Features

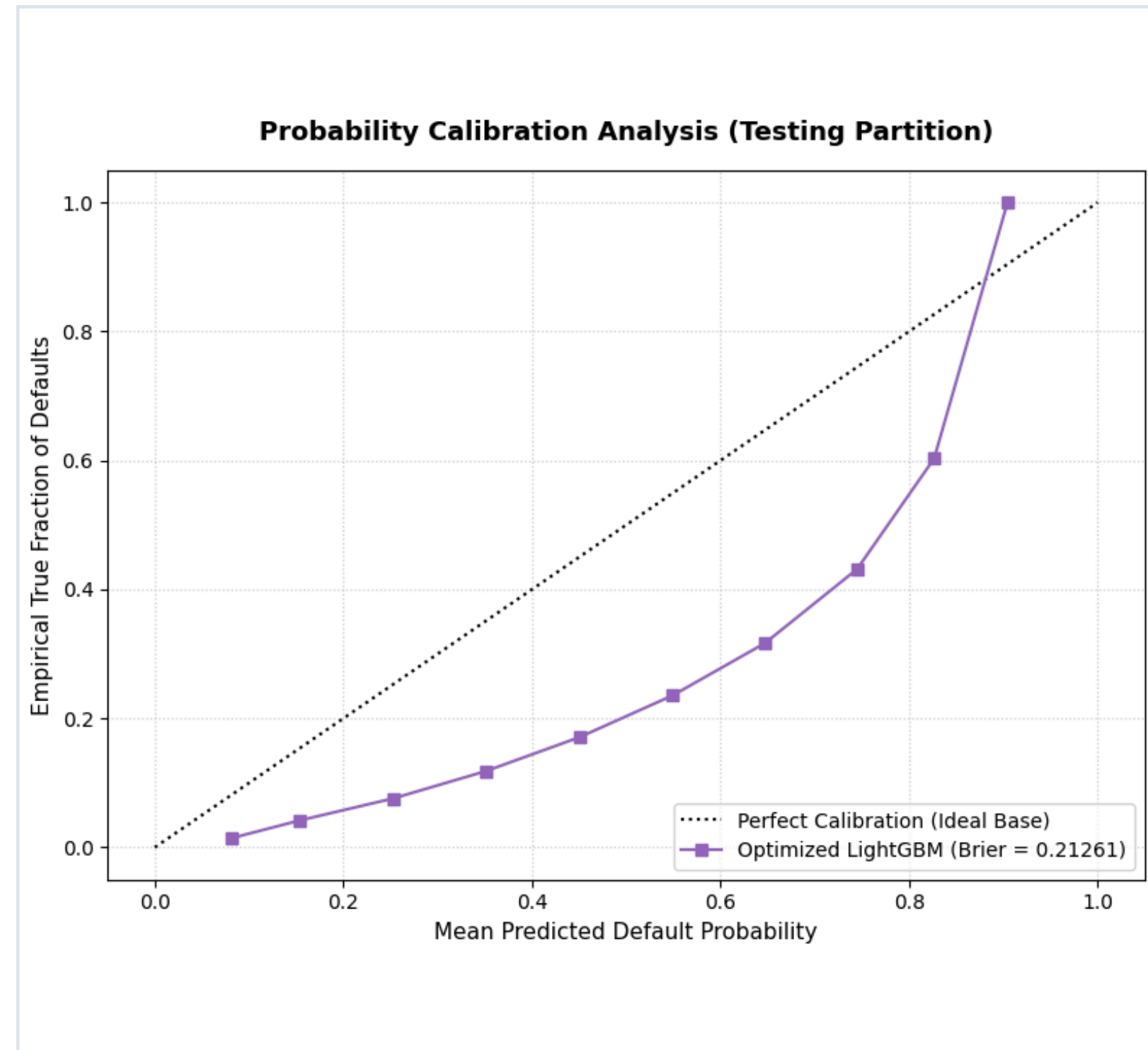
Rank	Feature	Mean SHAP	Min Impact	Max Impact
1	sub_grade	0.3782	-0.9613	+0.7451
2	Paidback period (term)	0.1690	-0.1716	+0.5617
3	dti	0.1254	-0.2644	+0.2943
4	funded_amnt	0.1109	-0.6414	+0.4120
5	acc_open_past_24mths	0.0922	-0.1906	+0.2771
6	annual_inc	0.0765	-0.1923	+0.3579
7	int_rate	0.0760	-0.3345	+0.3544
8	fico_range_low	0.0698	-0.2934	+0.1615
9	num_actv_rev_tl	0.0545	-0.1729	+0.2215
10	mths_since_recent_inq	0.0487	-0.0798	+0.2052
11	mort_acc	0.0480	-0.1705	+0.1027
Full audit ranked all 62 model features; the bottom ~9 features (e.g. acc_now_delinq, out_prncp, num_tl_30dpd) carry essentially zero SHAP impact, confirming the model concentrates its signal on a compact, interpretable set of origination-time drivers.				
12	avg_cur_bal	0.0462	-0.1525	+0.0727
13	home_ownership_MORTGAGE	0.0436	-0.0881	+0.0825

SHAP Waterfall: Explaining One Individual Decision



- For a single held-out applicant, the waterfall plot decomposes the model's prediction into per-feature contributions, starting from the population baseline $E[f(x)] = -0.227$.
This applicant's sub_grade contributes the largest single push toward a lower risk score (-0.76), while acc_open_past_24mths and num_actv_rev_tl push modestly upward.
Why this matters operationally: this is the mechanism that lets a loan officer explain to a real applicant — or a regulator — exactly why the model scored them the way it did, feature by feature.

Are the Model's Probabilities Trustworthy?



What this checks: if the model says "30% default probability" for a group of applicants, roughly 30% of them should actually default — that's perfect calibration (the dotted diagonal).

Finding: the Optimized LightGBM systematically under-predicts risk at low-to-mid probability bins and over-predicts at the highest bin (Brier score = 0.2126) — a common pattern for class-weighted boosting models, since `scale_pos_weight` distorts raw probability outputs even as it improves ranking.

Practical implication: the model is excellent for ranking applicants by relative risk (which is what ROC-AUC/AP measure), but its raw probability outputs would benefit from a post-hoc calibration step (e.g. Platt scaling or isotonic regression) before being used as literal default probabilities in pricing.

What We Learned About This Data and Problem

Leakage is the central risk in this domain, not an edge case. Two independent, non-obvious leakage sources (post-origination payment fields, then retrospective FICO fields) both produced models that looked excellent and would have failed in production. Forensic SHAP analysis — not accuracy alone — is what caught both.

Credit default is genuinely hard to predict from origination-time data. A realistic ROC-AUC of 0.72 is consistent with published industry benchmarks, and is far more valuable than the illusory 0.9999 from Round 1.

The strongest legitimate predictors are exactly the ones a human underwriter would expect: credit sub-grade, loan term, debt-to-income ratio, and recent credit activity — the model rediscovered sound underwriting logic rather than a leakage shortcut.

Limitations & Failure Cases

Calibration gap: raw probability outputs are not well-calibrated (Brier = 0.2126); the model ranks risk well but should not be read as literal default probabilities without post-processing.

Moderate Average Precision (0.396): at the minority-class level, roughly 6 in 10 flagged high-risk applicants would still be false alarms at typical operating thresholds — a real business cost to weigh against missed defaults.

Static, historical data: the 2007–2018 window includes the 2008 financial crisis; macroeconomic regime shifts are not modeled, so performance could drift in a new economic environment.

No formal fairness audit: features like `addr_state` and `home_ownership` carry demographic correlation risk that was not stress-tested for disparate impact in this iteration.

Concrete Improvements for a Future Iteration

Post-hoc probability calibration (Platt scaling / isotonic regression) before any probability is used for pricing or capital-reserve decisions.

Threshold-sensitive cost analysis: translate the ROC/PR curves into an explicit approve/reject decision threshold using real cost estimates for Type I vs. Type II errors, rather than stopping at the ranking metric.

Temporal validation: re-split by origination vintage (train on earlier years, test on later years) instead of a random stratified split, to pressure-test performance under genuine distribution shift.

Fairness and disparate-impact testing on protected-adjacent features (geography, home ownership) before any deployment consideration.

Formal leakage regression tests: the two-round leakage discovery in this project argues for codifying an automated "future-field" audit as a permanent, repeatable pipeline step, not a one-time manual investigation.

Thank You

Credit Risk Assessment — LendingClub Default Prediction

- 1.34M loans · 62 leakage-free features · Optimized LightGBM champion
- Test ROC-AUC 0.7248 · Average Precision 0.3961
- The leakage investigation: 0.9999 → 0.9519 → 0.7248